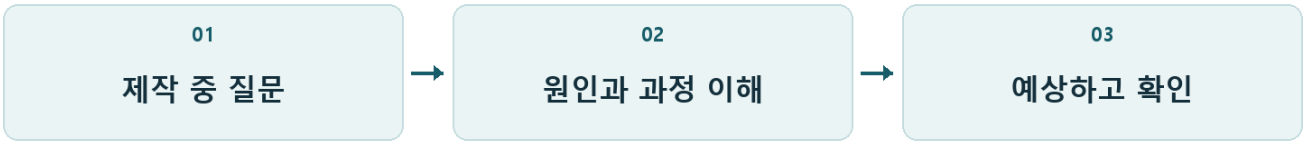
언리얼 엔진 5 렌더링 원리와 최적화

게임 배경 아티스트를 위한 기술 학습 시리즈

이 문서가 설명하는 것

모델과 재질, 빛이 화면으로 바뀌는 과정을 이해하고, 외관과 성능 문제의 원인을 구분하는 방법을 배웁니다. 기술 이름을 외우는 대신 제작 중의 질문 하나를 원리로 설명하고, 예상 결과가 있는 실험으로 확인합니다.



학습의 순서

렌더링 단계에서 시작해 재질과 빛, 에셋 처리, 시간적 화질, 성능 검증으로 연결합니다. 각 챕터의 첫 문장이 답의 중심입니다. 그림은 그 답을 이해하는 데 쓰고, 실습은 그 답이 장면에서 성립하는지 확인합니다.

실습과 자료의 상태

UE 5.8.4 데스크톱 프로젝트를 기준으로 합니다. CH01에는 직접 제작한 쇼윈도와 실제 캡처가 있습니다. CH02부터 CH26은 기본 레벨과 제작·검증 절차이며, 이 개정에서 새 예제 제작이나 성능 실측을 완료한 것은 아닙니다.

이미지의 구분

엔진 캡처, 직접 제작 개념도, 외부 렌더·진단 화면을 캡션에서 구분합니다. 외부 예시의 버전·수치를 현재 프로젝트의 결과로 옮기지 않습니다.

목차

렌더링 기초

01 렌더링 파이프라인 · 4
02 Forward와 Deferred · 10
03 GBuffer와 표면 정보 · 12
04 DBuffer와 데칼 · 14
05 Depth와 Early Z · 16
06 Overdraw와 화면 겹침 · 18

머티리얼과 빛

07 PBR과 재질 입력 · 20
08 BRDF와 반사 분포 · 22
09 Fresnel과 시선 각도 · 24
10 셰이더와 WPO · 26
11 Substrate와 물질 적층 · 28
12 Translucency와 합성 · 30
13 SSS와 표면 아래 산란 · 32

지오메트리와 텍스처

14 Draw Call과 제출 작업 · 34
15 Instancing과 반복 배치 · 36
16 LOD와 화면 크기 · 38
17 Nanite와 가상화 지오메트리 · 40
18 Culling과 가시성 · 42
19 Mipmap과 샘플링 · 44
20 Texture Streaming · 46

라이팅과 화면 품질

21 Lumen과 간접광 · 48
22 Virtual Shadow Maps · 50
23 TAA와 프레임 히스토리 · 52
24 TSR과 시간적 재구성 · 54

실전 성능 분석

25 CPU와 GPU 병목 · 56
26 GPU Profiler와 개선 검증 · 58

이 문서를 읽는 방법 · 3

종합 실습 · 60

용어집 · 61

심화와 적용 조건 · 67

출처 목록 · 68

이 문서를 읽는 방법

첫째 중심 질문과 답부터 읽습니다

각 챕터는 제작 중 겪는 질문 하나로 시작합니다. 먼저 답을 읽고, 그 답이 왜 성립하는지 이어지는 원인과 과정을 따라갑니다. 다른 주제의 예외는 마지막에 따로 둡니다.

둘째 그림이 보여 주는 차이를 찾습니다

그림의 설명을 읽고 비교할 위치와 변수를 찾습니다. 실제 렌더 그림은 외관의 사례이고 개념도는 관계의 설명입니다. 하나가 다른 하나의 측정 증거를 대신하지 않습니다.

셋째 실습 전에 결과를 예상합니다

실습 절차 다음의 예상 결과를 먼저 읽습니다. 무엇을 유지하고 무엇만 바꾸는지 확인합니다. 실제 결과가 다르면 해석 항목을 따라 조건과 가정을 점검합니다.

넷째 제작 결정으로 연결합니다

실습은 최종 목적이 아닙니다. 그 원리가 다른 건물·식생·재질·조명에서도 어떤 선택에 쓰이는지 읽고 자신의 장면에 적용합니다.

성능 비교의 공통 조건

기기·엔진·그래픽 API·출력 크기·카메라·노출·품질을 기록합니다. 셰이더 준비와 로딩을 끝낸 뒤 같은 경로를 반복합니다. FPS와 함께 작업 시간인 ms를 기록하고, 필요한 외관이 유지되는지 봅니다.

본문 밖으로 옮긴 내용

정의는 처음 필요한 자리에서 설명하고, 용어집은 재확인에 사용합니다. 수식·순열·모델별 입력·저장 형식·도구 버전은 본문 결론을 이해한 뒤 심화와 공식 자료에서 확인합니다.

01 렌더링 파이프라인

중심 질문

같은 장면인데 왜 진단 화면마다 보이는 것이 다를까?

먼저 알아둘 답

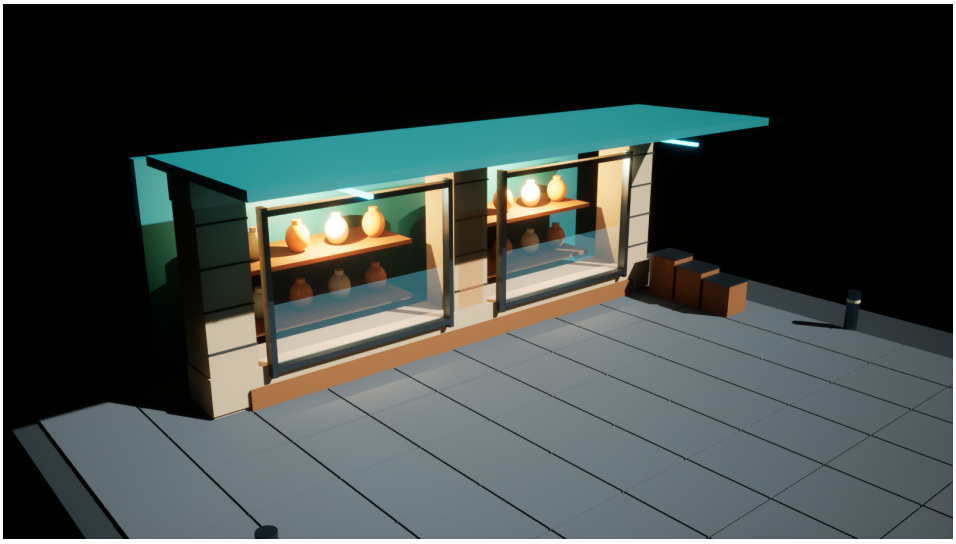
최종 화면은 한 번에 그린 사진이 아니라, 형태·표면·빛·합성의 결과를 이어 만든 이미지입니다.

왜 알아야 할까

화면이 이상할 때 메시, 재질, 조명, 후처리 중 어디를 먼저 조사할지 정하기 위해 배웁니다.

화면이 만들어지는 과정

CPU는 장면 상태를 바탕으로 그릴 작업을 준비하고 GPU는 그 명령에 따라 계산합니다. 먼저 메시의 정점 위치를 카메라 화면에 대응시키고, 삼각형이 덮는 위치를 찾습니다. 그 위치의 표면 정보에 빛과 그림자를 반영하고, 유리 같은 표면과 화면 효과를 합쳐 최종 색을 만듭니다.

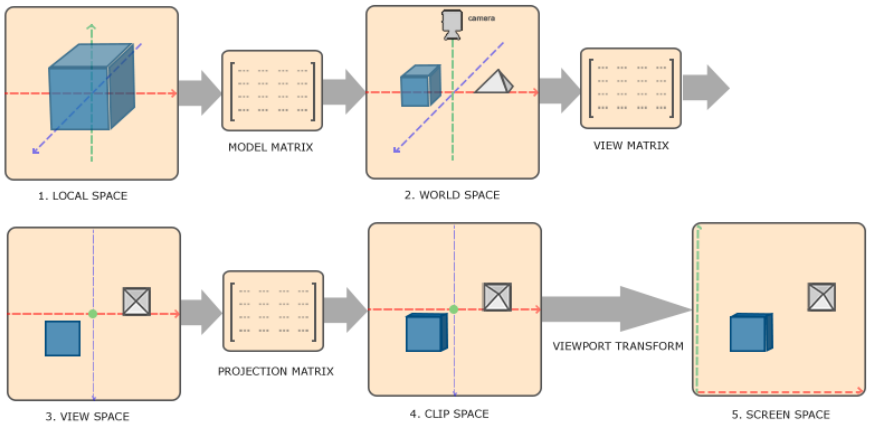


직접 제작·실제 UE 5.8.4 캡처. 이 쇼윈도에서 형태·표면·조명·후처리의 기여를 나눠 봅니다.

모델의 위치가 화면 위치가 되는 과정

형태를 이루는 정보

지오메트리는 물체의 위치와 모양을 정하는 기하 정보입니다. 메시의 표면은 삼각형으로 표현할 수 있고, 정점은 그 꼭짓점입니다. 정점 셰이더는 GPU에서 이 위치 등을 계산하는 프로그램입니다.



외부 참고 그림 · 모델의 로컬 공간부터 월드·카메라·클립·화면 공간까지 이어지는 좌표 변환입니다.

1→2에서는 큐브를 장면에 배치하고, 2→3에서는 카메라를 기준으로 장면을 해석합니다. 4→5에서는 투영한 결과가 화면 영역에 대응합니다. UE와 OpenGL의 축·깊이 규약은 같지 않지만 좌표 기준을 바꾸는 목적은 같습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [37]

원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

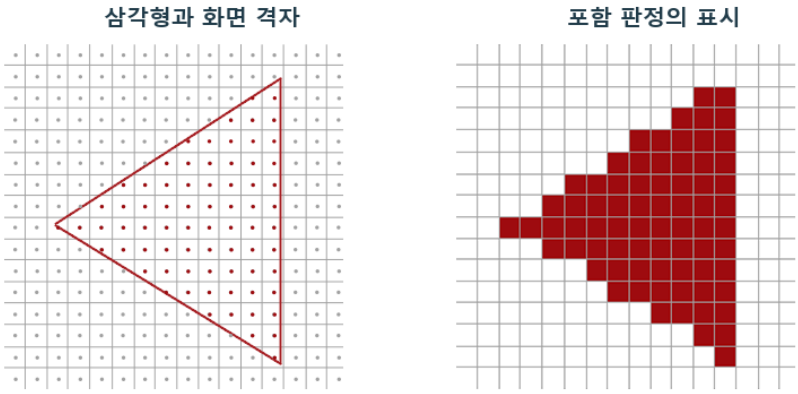
좌표가 달라지는 이유

모델 내부의 꼭짓점 위치에 배치의 이동·회전·크기를 반영하면 월드 위치가 됩니다. 이후 카메라를 기준으로 표현하고 투영해 화면 위치를 구합니다. 예를 들어 회전·확대 없이 원점을 X=10에 놓으면 로컬 X=1인 점의 월드 X는 11입니다.

아트 작업에서의 구분

위치 변환은 모델을 어디에 보이게 할지 정합니다. UV는 표면의 어느 텍스처 위치를 읽을지, 법선은 표면이 어느 방향을 향하는지 정합니다. 위치·무늬·반사의 방향을 서로 다른 정보로 구분합니다.

삼각형 내부의 화면 값을 만드는 과정



외부 비교 그림 · 삼각형과 화면 격자, 그리고 포함 판정을 표시한 결과를 나란히 배치했습니다.

왼쪽의 매끈한 삼각형 경계와 오른쪽의 격자에 맞춘 계단형 경계를 비교합니다. 이 그림은 래스터화와 앨리어싱의 관계를 설명하며 TAA 또는 TSR의 전후 결과가 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [38]
원본 전체 유지 · 비율 유지 · CC BY 4.0
이용 조건

래스터화

래스터화는 화면으로 옮긴 삼각형이 어느 샘플 위치를 덮는지 판정하는 과정입니다. 샘플은 검사하는 화면 위치입니다. 삼각형의 격자 포함 여부와 가려짐 검사를 거쳐 최종 화면에 기여할 후보를 찾습니다.

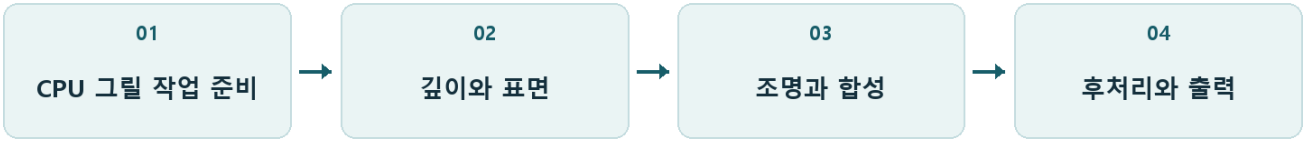
보간

보간은 알려진 값 사이의 중간값을 구하는 것입니다. 꼭짓점에 있는 UV 등을 이용해 삼각형 내부 위치의 값을 계산하므로, 가운데에도 텍스처를 연속적으로 붙일 수 있습니다. 원근 투영에서는 이에 맞는 보정이 적용됩니다.

왜 형상과 텍스처를 구분할까

텍스처는 삼각형 안의 무늬를 세밀하게 만들 수 있지만 바깥 실루엣의 곡선을 직접 늘리지는 않습니다. 표면 무늬가 거친지, 형상 윤곽이 각진지에 따라 수정할 데이터를 고릅니다.

화면 계산을 단계로 나누어 읽기



직접 제작 개념도. 작업 의존 관계이며 GPU 전체 실행 순서표는 아닙니다.

준비와 실행

스레드는 명령을 이어 실행하는 흐름입니다. 게임 스레드는 주로 장면 상태를 갱신하고, 렌더 스레드는 CPU에서 그릴 명령을 준비합니다. RHI는 Render Hardware Interface로 엔진 명령을 DirectX·Vulkan 같은 그래픽 API에 연결하는 계층입니다. GPU는 제출받은 작업을 실행합니다.

깊이와 표면 데이터

버퍼는 계산 중 쓸 데이터를 보관하는 메모리 공간입니다. 깊이는 앞뒤 관계를, GBuffer는 Deferred 조명에 필요한 표면 정보를 담습니다. 렌더 패스는 이런 계산을 나눠 수행하는 작업 단위입니다. 하나의 메시도 깊이·표면·그림자 작업에 각각 참여할 수 있습니다.

빛과 합성

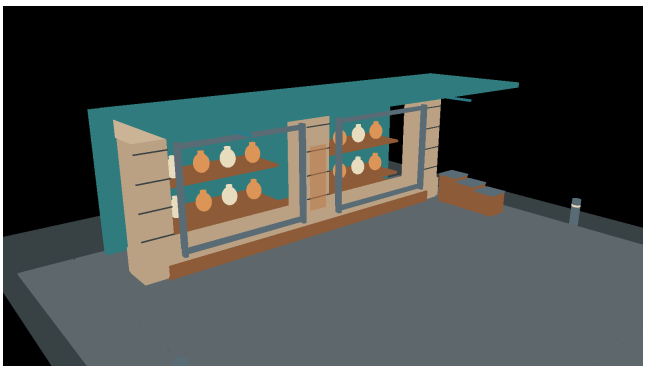
표면의 방향과 반사 특성에 조명·그림자를 반영합니다. 뒤가 비치는 표면은 배경과 합성하고, 밝은 부분을 퍼뜨리는 Bloom이나 색을 조정하는 후처리를 거쳐 표시합니다.

이 단계 구분을 쓰는 곳

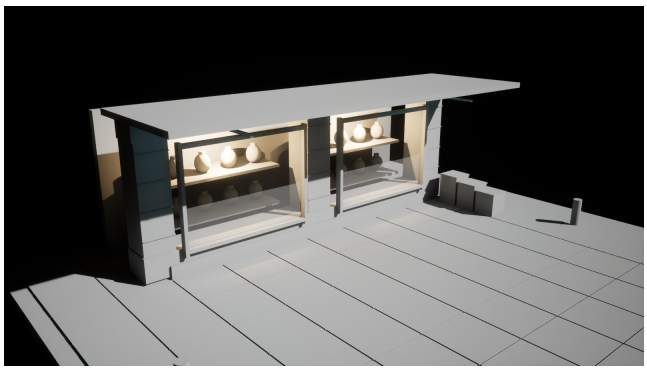
따라서 최종 색이 밝다는 사실만으로 Base Color가 밝다고 판단할 수 없습니다. 조명이나 발광, Bloom이 밝게 보이게 했을 수도 있습니다. 진단 화면은 이 중 특정 정보만 꺼내 보여 주므로 서로 다르게 보이는 것이 정상입니다.

표면색과 조명 결과를 나누어 보기

Base Color



Lighting Only



직접 제작·같은 카메라의 엔진 캡처. 왼쪽은 기본 표면색, 오른쪽은 조명 기여를 읽는 진단입니다. 두 화면을 더하면 Lit가 된다는 분해식은 아닙니다.

같은 벽을 따라 봅니다

최종 화면에서 밝은 벽이 Base Color에서도 밝은지 확인합니다. Base Color는 재료 입력을, Lighting Only는 조명 상태를 읽는 데 사용합니다. 각 모드는 용도에 맞게 표시하므로 최종 화면과 외관이 같을 필요가 없습니다.

다음 조사 대상을 고릅니다

입력 표면색부터 이상하면 재질을, 표면색은 정상인데 최종 밝기만 이상하면 조명·노출·후처리를 다음 조사 대상으로 고릅니다.

확인 범위

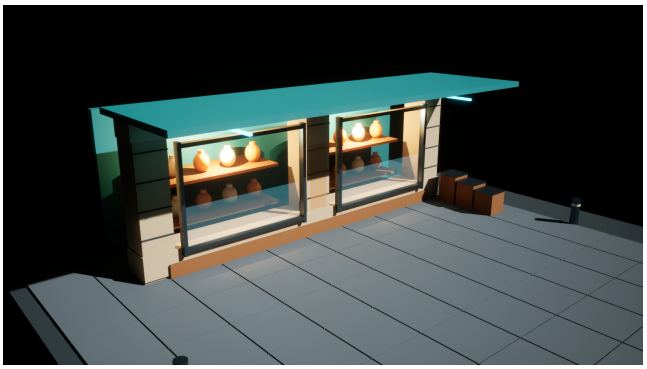
불투명 표면 진단만으로 유리 합성과 모든 후처리를 설명할 수 없습니다. 유리는 CH12, GBuffer는 CH03에서 각각 질문을 분리해 다룹니다.

예상하고 확인하는 첫 실습

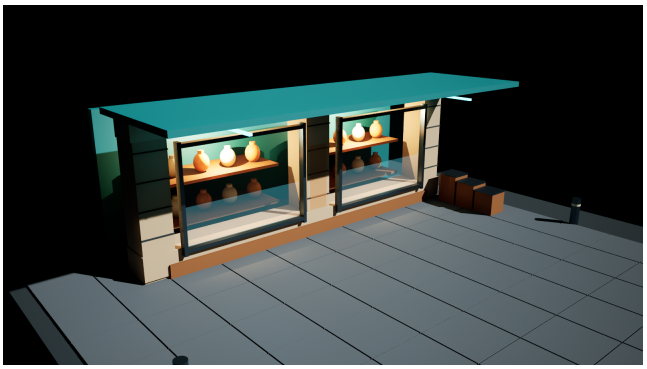
예제 목표

후처리의 변화와 기본 표면 입력을 구분합니다.

Bloom 적용



Bloom 해제



직접 제작·동일 구도의 엔진 비교. 밝은 바의 주변 번짐을 봅니다. 캡처 중의 시간은 성능 실측으로 사용하지 않습니다.

실습 절차

CH01 쇼윈도에서 같은 카메라를 유지합니다. Lit와 Base Color를 먼저 비교한 뒤 Bloom만 끄고 Lit와 Base Color를 다시 봅니다. 노출과 재질 값은 그대로 둡니다.

예상 결과와 이유

Bloom을 끄면 밝은 바 주변의 번짐이 줄어들지만, 기본 표면색은 같은 상태로 남을 것입니다. 이는 화면 뒤쪽 단계의 변화가 앞쪽 표면 입력을 바꾸지 않는다는 예상입니다.

결과가 다를 때

번짐과 표면색이 함께 바뀌면 Bloom 외의 설정이나 노출이 달라졌는지 먼저 확인합니다. 진단 화면이 달라진 이유를 찾는 것이 이번 실습의 목적이며, 이 비교만으로 성능 향상까지 판단하지 않습니다.

남길 결론

최종 화면은 한 번에 그린 사진이 아니라, 형태·표면·빛·합성의 결과를 이어 만든 이미지입니다.

레벨 /Game/RenderingLab/Chapters/Part01_RenderingBasics/CH01_RenderingPipeline/Maps/L_CH01_RenderingPipeline

02 Forward와 Deferred

중심 질문

왜 렌더러를 바꾸면 같은 에셋의 표현과 비용 조건이 달라질까?

먼저 알아둘 답

두 방식은 표면과 빛을 결합하는 시점이 달라, 중간 데이터와 사용할 수 있는 기능의 조건도 달라집니다.

왜 알아야 할까

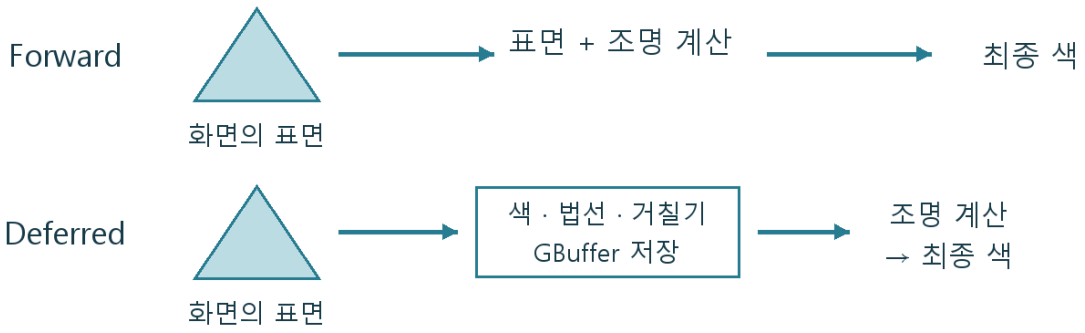
목표 플랫폼과 필요한 표현을 기준으로 렌더러를 선택하고, 그 선택에 맞는 에셋 제작 조건을 이해하기 위해 배웁니다.

왜 이런 결과가 생길까

Forward는 표면을 그리는 계산에서 조명을 함께 평가합니다. Deferred는 먼저 화면에 보이는 표면의 색·방향·거칠기 등을 저장하고, 조명 단계가 그 정보를 읽어 빛을 계산합니다. Deferred는 표면 정보를 다시 사용할 수 있지만, 저장하고 읽는 작업도 필요합니다. UE의 Forward 역시 영향을 주는 광원을 선별하며 무조건 모든 광원을 전부 계산하지 않습니다.

제작 중에는 이렇게 나타납니다

조명이 많은지뿐 아니라 필요한 AA, 반사, 재질 기능과 중간 데이터의 비용까지 선택에 영향을 줍니다. 이름만 보고 어느 쪽이 항상 빠르다고 정할 수 없는 이유가 여기에 있습니다.



차이의 중심은 조명 전에 표면 데이터를 따로 저장하고 읽는가입니다.

직접 제작 개념도. 표면과 조명을 결합하는 시점 및 중간 저장의 차이입니다. 실제 프레임의 모든 패스를 나열한 도식은 아닙니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

같은 장면에서 표면 처리와 조명 작업이 어디에 배치되는지 확인합니다.

실습 절차

프로젝트 복사본에서 Forward Shading을 바꾸고 재시작합니다. 공통 지원하는 불투명 재질과 그림자 없는 이동형 라이트만 둔 작은 장면을 준비합니다. 해상도와 지원 기능을 맞춘 뒤 광원 수를 한 단계씩 늘려 패스 구성을 비교합니다.

예상 결과와 이유

Deferred에서는 표면을 기록한 뒤 별도 조명 작업이 이어지고, Forward에서는 표면을 그리는 작업에 조명 계산이 포함되는 구조를 확인할 수 있을 것입니다.

결과를 어떻게 해석할까

패스 이름 하나의 시간만 직접 맞대지 않습니다. 계산이 서로 다른 작업으로 나뉘므로 전체 GPU 시간과 화면 결과를 함께 기록합니다. 설정 변경 후 기능이 빠졌다면 동일한 화질의 성능 비교로 사용하지 않습니다.

실제 제작에서의 선택

VR의 경계 품질이나 반사 표현처럼 필요한 기능을 먼저 적고, 그 조건을 만족하는 렌더러 안에서 비용을 비교합니다.

이해한 뒤 확인할 조건

이 장은 데스크톱 렌더링의 기본 구조를 설명합니다. 모바일 경로와 플랫폼별 기능 지원표는 공식 문서에서 별도로 확인합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[01] Rendering Settings

[04] Forward Shading Renderer

03 GBuffer와 표면 정보

중심 질문

벽이 번들거릴 때 왜 최종 색보다 Roughness 화면을 먼저 볼까?

먼저 알아둘 답

GBuffer는 조명을 계산하기 전의 표면 정보를 보관하므로, 재질 입력 문제와 조명 결과를 나눠 볼 수 있습니다.

왜 알아야 할까

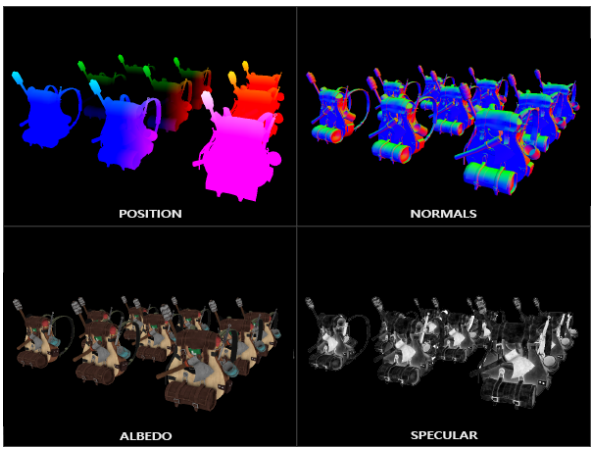
반사가 어색할 때 재질을 수정해야 하는지 조명을 수정해야 하는지 판단하기 위해 배웁니다.

왜 이런 결과가 생길까

Deferred 렌더러는 보이는 불투명 표면의 기본 색, 표면 방향인 법선, 거칠기 같은 정보를 기록합니다. 조명 단계는 이 값을 읽어 반사의 색과 분포를 계산합니다. 같은 표면 정보라도 광원의 위치와 밝기가 달라지면 최종 색은 달라집니다. Buffer Visualization은 그 입력 정보를 따로 확인하는 통로입니다.

제작 중에는 이렇게 나타납니다

Roughness는 반사가 퍼지는 정도를 조절하는 값입니다. 벽의 반사가 좁고 강해 보일 때 거칠기가 의도보다 낮은지 먼저 보면, 라이트를 임의로 약하게 만들어 재질 오류를 가리는 일을 줄일 수 있습니다.



외부 시각화 이미지 · 같은 장면의 위치·법선·Albedo·Specular를 분리해 표시한 GBuffer 예시입니다.

왼쪽 위는 위치에 따라, 오른쪽 위는 표면 방향에 따라 색이 달라집니다. 아래쪽은 재질의 색과 반사 입력입니다. 진단용 색을 최종 외관이나 UE 채널 구성으로 읽지 않습니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [39]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

거칠기 변경과 광원 변경이 서로 다른 진단 화면에 나타나는지 확인합니다.

실습 절차

같은 벽의 Material Instance 두 개를 만들고 Roughness만 0.2와 0.8로 둡니다. Material Instance는 원본 재질의 구조를 공유하며 설정값을 바꾸는 변형입니다. 같은 구도에서 Roughness, Base Color, Lit 순서로 비교합니다.

예상 결과와 이유

Roughness 진단은 달라지고 Base Color는 유지되며, Lit에서는 반사의 퍼짐이 달라질 것입니다. 다음으로 라이트 위치만 바꾸면 Lit는 달라져도 두 표면 입력은 유지될 것입니다.

결과를 어떻게 해석할까

첫 비교는 재질 입력이 결과에 미치는 영향을, 두 번째 비교는 같은 입력에 조명이 미치는 영향을 보여 줍니다. 진단 표시색을 실제 저장 바이트 배열로 해석할 필요는 없습니다.

실제 제작에서의 선택

입력 진단이 의도와 다르면 텍스처·값·연결을 고칩니다. 입력이 맞으면 반사 환경과 광원, 노출을 조사합니다.

이해한 뒤 확인할 조건

GBuffer의 채널과 저장량은 렌더러·플랫폼·Substrate 설정에 따라 달라집니다. 해상도와 메모리 비용의 관계는 이 실험과 별개의 성능 확인 항목입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[05] Programming with Substrate GBuffer Formats

04 DBuffer와 데칼

중심 질문

데칼은 왜 벽 그림처럼 빛에 반응하고, 옆 소품에도 묻을까?

먼저 알아둘 답

DBuffer 데칼은 투영 범위 안의 표면 정보를 바꾸고, 바뀐 표면이 이후 조명 계산에 참여합니다.

왜 알아야 할까

반복 벽에 얼룩과 손상을 더하면서 투영 오류와 불필요한 처리 범위를 제어하기 위해 배웁니다.

왜 이런 결과가 생길까

데칼 Actor의 박스는 그림을 적용할 3D 범위를 정합니다. 엔진은 깊이 정보로 수신 표면의 위치를 찾고, 박스 안에 해당하는 데칼 값을 준비합니다. DBuffer는 그 색·거칠기·법선 변경을 잠시 담는 버퍼입니다. 벽의 Base Pass가 이를 읽어 표면에 반영하므로 단순히 화면 위에 붙인 스티커와 다르게 조명에 반응합니다.

제작 중에는 이렇게 나타납니다

박스는 원하는 벽 이름만 골라 투영하는 것이 아닙니다. 깊이가 과하면 그 안에 들어간 소품도 수신 조건에 따라 영향을 받습니다. 보이는 얼룩 크기와 투영 공간의 크기를 구분해야 합니다.



공식 비교 이미지 · 같은 돌벽에 데칼을 적용하기 전과 후입니다. UE 5.5 문서의 사례입니다.

반복되는 벽돌 위에 생긴 손상과 표면 변화가 어디까지 이어지는지 봅니다. 데칼 수나 성능의 통제 실험은 아니므로 투영 면적·겹침의 비용은 실습에서 따로 측정합니다.

© Epic Games · 공식 문서 · 출처 [46]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 원저작권과 원문 출처 표시는 유지합니다

예상한 결과를 장면에서 확인하기

이 예제의 목표

벽의 얼룩을 유지하면서 박스 깊이만 줄여 잘못 묻는 소품을 제외합니다.

실습 절차

벽 앞에 작은 소품을 놓고 하나의 얼룩 데칼을 투영합니다. 먼저 큰 박스에서 잘못 묻는 표면을 찾고, 데칼의 전면 크기는 유지한 채 박스 깊이만 줄입니다.

예상 결과와 이유

벽의 얼룩은 남고 박스 밖으로 빠진 소품의 얼룩은 사라질 것입니다. 이후 라이트 방향만 바꾸면 얼룩을 포함한 표면이 빛에 함께 반응할 것입니다.

결과를 어떻게 해석할까

소품에 계속 묻으면 실제 박스 위치와 수신 설정을 확인합니다. 색만 비교하지 말고 거칠기나 법선 변경도 확인해야 물기·스크래치 데칼의 효과를 놓치지 않습니다.

실제 제작에서의 선택

원하는 표면을 충분히 덮는 최소한의 투영 범위를 사용합니다. 넓은 중첩은 화면에서 같은 영역을 반복 처리할 수 있으므로 큰 박스를 관성적으로 유지하지 않습니다.

이해한 뒤 확인할 조건

Receives Decals는 물체의 수신 여부, Decal Response는 재질이 받을 변경을 정합니다. 다른 데칼 경로와 적용 순서는 공식 자료의 심화 항목입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

05 Depth와 Early Z

중심 질문

앞에 벽이 있으면 뒤의 비싼 재질 계산도 하지 않을까?

먼저 알아둘 답

깊이 검사는 가려짐을 판정하고, 그 판정을 재질 계산보다 일찍 할 수 있을 때 뒤 표면의 불필요한 계산을 줄입니다.

왜 알아야 할까

실내·밀집 건축·식생에서 가림이 실제로 어떤 일을 줄이는지 이해하기 위해 배웁니다.

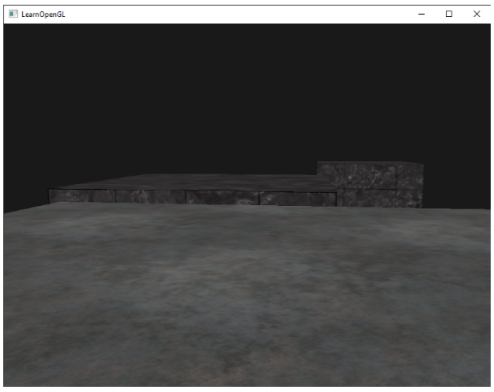
왜 이런 결과가 생길까

화면의 한 위치에 여러 불투명 표면이 겹치면 가까운 표면이 남아야 합니다. 깊이 버퍼는 이 비교에 쓸 깊이를 보관합니다. Early Z는 깊이로 탈락할 후보를 비싼 픽셀 계산 전에 제외하는 처리입니다. Depth Prepass는 깊이를 먼저 그려 준비하는 별도 작업이며, Early Z 자체와 같은 뜻은 아닙니다.

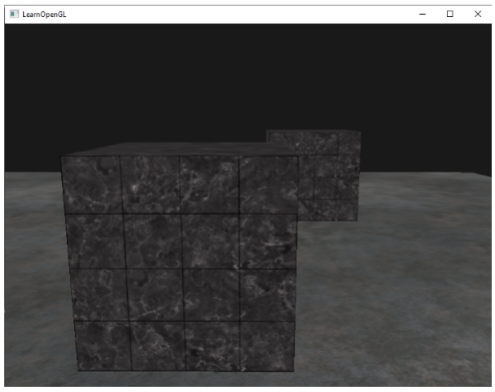
제작 중에는 이렇게 나타납니다

깊이를 먼저 준비하면 뒤의 재질 계산을 줄일 기회가 생기지만, 앞서 깊이를 그리는 일도 비용입니다. 따라서 최종 화면에서 가려졌다는 사실과 GPU 일이 얼마나 줄었는지는 별개입니다.

깊이 항상 통과



앞쪽 깊이로 검사



외부 비교 이미지 · 왼쪽은 깊이 비교를 항상 통과, 오른쪽은 앞쪽 깊이를 선택한 OpenGL 예시입니다.

왼쪽에서 바닥이 큐브를 잘못 덮는 부분과 오른쪽의 올바른 가림을 비교합니다. 깊이 검사의 필요성을 보여 주며, Early Z로 몇 ms를 절약했는지 입증하는 이미지는 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [47]
동일 범위의 원본을 나란히 배치 · 표제 추가 · CC BY 4.0
이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

가림 범위가 깊이 표시와 뒤쪽 재질 처리 시간에 미치는 영향을 구분합니다.

실습 절차

비교용 전통 불투명 메시의 뒤 패널에 계산이 많은 재질을 둡니다. 카메라를 고정하고 앞의 단순한 가림판으로 패널을 절반, 대부분 가리는 두 상태를 만듭니다. Prepass 설정은 유지하고 Scene Depth와 Base Pass 시간을 봅니다.

예상 결과와 이유

Scene Depth에는 가림판이 앞면으로 보일 것입니다. 조기 깊이 제거가 유효하고 뒤 패널의 픽셀 계산 비중이 크다면, 더 많이 가린 상태에서 해당 계산 시간이 줄 것으로 예상합니다.

결과를 어떻게 해석할까

시간이 그대로면 뒤 표면이 원래 싸거나, 다른 일이 지배적이거나, 조기 제거 조건이 다를 수 있습니다. 깊이 표시가 맞는다는 것만으로 최적화 효과를 입증하지 않습니다.

실제 제작에서의 선택

불투명 가림을 활용하되 Prepass만 늘리면 빨라진다고 결정하지 않습니다. 가려진 픽셀 계산을 줄인 이득과 깊이 준비 비용을 함께 봅니다.

이해한 뒤 확인할 조건

Masked, 깊이를 바꾸는 Pixel Depth Offset, 반투명은 조건이 다릅니다. 먼저 불투명 재질에서 기본 관계를 이해한 뒤 별도로 비교합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[03] Introduction to Performance Profiling and Configuration

[06] Decal Materials

[30] Visibility and Occlusion Culling

06 Overdraw와 화면 겹침

중심 질문

삼각형을 줄인 풀 카드가 왜 더 느릴 수 있을까?

먼저 알아둘 답

풀의 비용은 삼각형 수뿐 아니라 잎이 없는 영역과 겹친 층에서 반복하는 화면 계산에도 달려 있습니다.

왜 알아야 할까

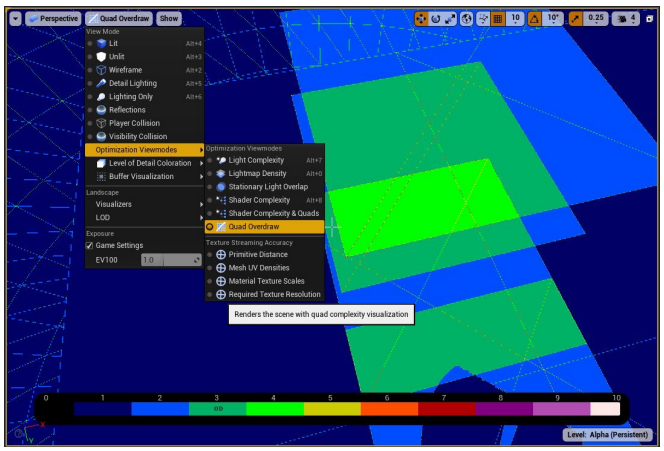
식생과 파티클의 모양을 다듬을 때 폴리곤 수와 화면 낭비 사이의 균형을 잡기 위해 배웁니다.

왜 이런 결과가 생길까

큰 사각형 카드에 작은 잎을 그리면 사각형 안에서 어느 부분을 남길지 마스크를 검사해야 합니다. 빈 영역이 넓거나 카드가 여러 겹이면 화면의 비슷한 위치를 반복 처리할 수 있습니다. Overdraw는 이런 중복 처리와 관련된 개념입니다. 잎 윤곽대로 메시를 다듬으면 삼각형은 늘어도 검사할 빈 영역은 줄어듭니다.

제작 중에는 이렇게 나타납니다

가까이서 화면을 크게 덮는 식생에서는 삼각형 몇 개를 아끼는 것보다 카드의 빈 공간을 줄이는 편이 유리할 수 있습니다. 반대로 지나치게 세분한 형상은 정점과 작은 삼각형 처리 부담을 늘립니다.



외부 진단 화면 · Android Developers가 소개한 Unreal Editor의 Quad Overdraw 시각화입니다.

겹친 영역의 진단색과 아래 범례를 같이 봅니다. 작은 화면 묶음에서의 처리 비효율을 찾는 화면이며 실제 GPU 시간은 아닙니다. 원본은 이전 에디터 UI로 UE 5.8의 메뉴 배치는 다를 수 있습니다.

Android Developers · Google · 출처 [48]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · CC BY 4.0
이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

삼각형을 더 쓴 카드가 빈 화면 영역의 처리를 줄일 수 있는지 확인합니다.

실습 절차

같은 잎 텍스처를 쓰는 큰 사각형 카드와 윤곽을 다듬은 카드를 만듭니다. 잎의 실제 크기·배치·재질·그림자는 같게 두고 가까운 구도에서 비교합니다. Shader Complexity와 가능한 Quad Overdraw를 본 뒤 진단을 끄고 GPU 시간을 기록합니다.

예상 결과와 이유

윤곽을 다듬은 버전은 빈 영역의 진단 부담이 줄 것으로 예상합니다. 픽셀 처리가 병목이라면 삼각형이 늘어도 실행 시간이 줄 수 있습니다.

결과를 어떻게 해석할까

진단색 감소는 낭비 영역을 찾는 단서입니다. 실행 시간과 같은 단위가 아니므로 비용 감소는 별도로 확인합니다. 원거리에서도 같은 결론인지 다시 봅니다.

실제 제작에서의 선택

메시를 무조건 사각형으로 줄이지 말고, 보이는 잎 면적에 비해 빈 영역이 얼마나 큰지 먼저 확인합니다.

이해한 뒤 확인할 조건

Quad Overdraw는 작은 화면 묶음의 처리 비효율도 보여 줍니다. 일반 Overdraw와 완전히 같은 지표는 아니며 Nanite는 전용 시각화를 함께 사용합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[07] Viewport Modes

[19] Nanite Virtualized Geometry

07 PBR과 재질 입력

중심 질문

조명을 바꿔도 금속과 페인트가 재료답게 보이려면 무엇을 입력해야 할까?

먼저 알아둘 답

재료의 고유 반응을 입력하고, 장면의 빛과 그림자는 조명 계산에 맡겨야 합니다.

왜 알아야 할까

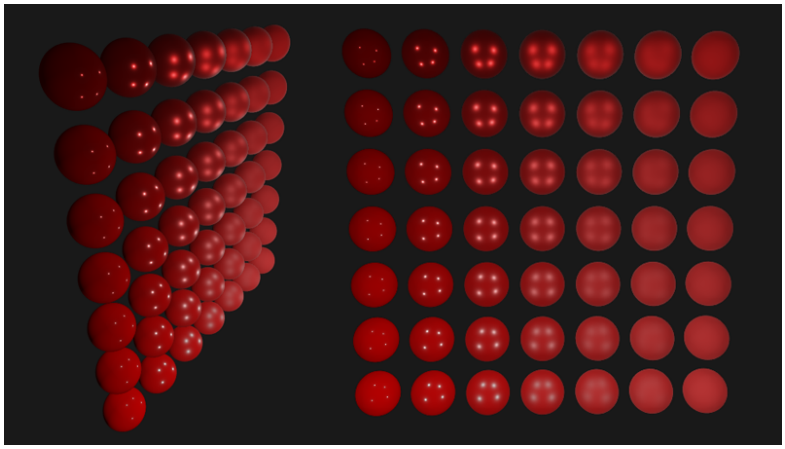
한 조명에서만 그럴듯한 텍스처 대신 여러 환경에 쓸 수 있는 배경 재질을 만들기 위해 배웁니다.

왜 이런 결과가 생길까

PBR은 빛과 물질의 상호작용을 물리적으로 그럴듯하게 근사하는 제작 방식입니다. 비금속의 Base Color는 주로 확산 반사의 색을, 금속의 Base Color는 금속 반사의 색을 정합니다. Metallic은 어느 재료 반응을 사용할지, Roughness는 반사가 얼마나 퍼질지 정합니다. 이 반응에 장면의 빛이 들어와 최종 외관이 생깁니다.

제작 중에는 이렇게 나타납니다

텍스처에 특정 광원의 하이라이트나 방향성 그림자를 그려 넣으면, 라이트가 움직여도 그 흔적이 남습니다. 도장과 벗겨진 철은 밝고 어두운 정도만 다른 것이 아니라 서로 다른 재료로 나눠 입력해야 합니다.



외부 렌더 이미지 · 금속성과 거칠기를 달리한 구들을 두 시점에서 비교합니다.

오른쪽 배열의 가로 방향에서 거칠기가 변할 때 작은 반사점이 넓어지는 모습을 봅니다. 세로 방향의 금속성 변화도 함께 비교합니다. UE와 광원·톤매핑이 같은 비교 자료는 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [40]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

금속성·거칠기 입력과 광원 이동의 역할을 나누어 관찰합니다.

실습 절차

같은 구에 Metallic 0과 1, Roughness 0.2와 0.8의 조합을 만듭니다. 처음에는 고정 조명에서 네 구를 비교하고, 이후 재질을 유지한 채 넓은 광원의 방향만 바꿉니다.

예상 결과와 이유

거칠기 변화는 반사의 퍼짐을, 금속성 변화는 확산·스펙큘러 색 반응을 바꿀 것입니다. 광원을 움직이면 하이라이트도 움직여야 합니다.

결과를 어떻게 해석할까

기본 색에 밝은 얼룩이 고정돼 남는다면 구워 넣은 조명 흔적인지 살핍니다. 금속이 검게 보이면 Metallic부터 임의로 낮추기 전에 반사할 환경이 있는지 확인합니다.

실제 제작에서의 선택

금속·도장·녹을 재료로 구분한 뒤 입력을 정합니다. 외관이 어색할 때 텍스처와 조명을 동시에 고치지 않습니다.

이해한 뒤 확인할 조건

순수 재료의 Metallic은 0 또는 1을 기준으로 하고 경계의 필터링 등에는 중간값이 생깁니다. sRGB·데이터 채널 패킹은 입력 저장의 심화 항목으로 분리합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[08] Physically Based Materials

[09] Using Fresnel

08 BRDF와 반사 분포

중심 질문

같은 밝기의 재질인데 왜 하이라이트의 폭과 모양이 다를까?

먼저 알아둘 답

빛은 모든 방향으로 똑같이 반사되지 않으며, 표면의 미세한 방향 분포가 반사의 모양을 바꿉니다.

왜 알아야 할까

거칠기와 광원 중 무엇을 조절해야 원하는 재질감이 나오는지 판단하기 위해 배웁니다.

왜 이런 결과가 생길까

BRDF는 들어오는 빛이 관찰 방향으로 얼마나 반사되는지 나타내는 함수입니다. 거친 표면은 여러 방향의 작은 면을 가진 것으로 근사할 수 있습니다. 거칠기가 높아지면 반사가 넓은 방향으로 퍼져 하이라이트가 넓어집니다. 넓은 광원 역시 여러 방향에서 빛을 보내므로 하이라이트 크기에 영향을 줍니다.

제작 중에는 이렇게 나타납니다

넓은 하이라이트를 봤다고 재질이 반드시 거칠다고 단정할 수 없습니다. 광원 크기와 표면 방향도 같은 외관에 관여합니다. 서로 다른 원인을 나눠 조절해야 재질을 다른 장면에 옮겨도 의도가 유지됩니다.



외부 개념도 · 미세한 표면 방향의 차이가 반사광의 퍼짐에 영향을 주는 모습입니다.

거친 표면에서는 여러 방향으로, 매끈한 표면에서는 더 모인 방향으로 반사하는 차이를 봅니다. 확대된 개념도이며 실제 메시의 모든 미세 면을 직접 만드는 뜻은 아닙니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [41]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

거칠기와 광원 크기를 따로 바꿔 비슷한 하이라이트 폭의 다른 원인을 확인합니다.

실습 절차

구 하나와 Rect Light를 둡니다. 먼저 광원 크기·위치·밝기를 고정하고 Roughness만 바꿉니다. 원래 거칠기로 돌린 뒤 광원의 가로·세로 크기만 바꿉니다.

예상 결과와 이유

첫 비교에서는 재질의 반사 분포가, 두 번째에서는 반사에 나타나는 광원의 범위가 달라질 것입니다. 둘 다 하이라이트가 넓어질 수 있지만 원인은 다릅니다.

결과를 어떻게 해석할까

하이라이트 폭뿐 아니라 선명도와 주변 분포를 함께 봅니다. 밝기와 노출까지 동시에 바꾸면 두 원인을 구분하기 어려워집니다.

실제 제작에서의 선택

재질 반응을 맞출 때는 고정된 조명 환경을 기준으로 거칠기를 조정하고, 장면의 연출을 위해 광원 크기를 조정합니다.

이해한 뒤 확인할 조건

BRDF의 수식과 미세면 가림 항은 보충 주석에서 읽습니다. 이 장의 목표는 수식 암기보다 광원과 재질의 역할을 구분하는 것입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[08] Physically Based Materials

[10] Overview of Substrate Materials

[31] Real Shading in Unreal Engine 4 · Brian Karis

09 Fresnel과 시선 각도

중심 질문

물이나 유리는 왜 비스듬히 볼 때 반사가 강해질까?

먼저 알아둘 답

반사되는 빛의 비율은 입사각에 따라 달라지며, 비금속은 스치는 각도에서 반사가 강해집니다.

왜 알아야 할까

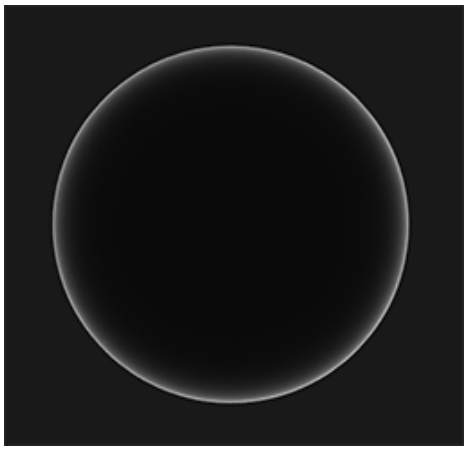
물·유리·코팅의 각도별 외관을 만들고 물리 반사와 의도적인 림 효과를 구분하기 위해 배웁니다.

왜 이런 결과가 생길까

Fresnel은 경계면에서 빛이 얼마나 반사되는지 각도와 물질 특성에 따라 설명합니다. 표면을 정면에서 보는 조건과 거의 스치듯 보는 조건의 반사율이 같지 않습니다. 실제 화면에서는 광원의 분포와 거칠기도 함께 작용하므로 가장자리가 언제나 밝은 선으로 보인다는 뜻은 아닙니다.

제작 중에는 이렇게 나타납니다

PBR 재질의 반사에는 이미 각도 의존성이 포함됩니다. 여기에 Fresnel 노드를 Emissive로 더하면 물리 반사뿐 아니라 광원 없이도 밝아지는 별도의 연출을 만든 것입니다.



외부 시각화 이미지 · 구의 중심과 실루엣 부근에서 Fresnel 반사 계수가 달라지는 예입니다.

중심보다 테두리가 밝습니다. 곡면의 테두리에서 시선이 표면에 더 비스듬하게 닿기 때문입니다. 물체의 완성 렌더가 아닌 반사 계수의 시각화입니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [41]
원본 전체 유지 · 비율 유지 · CC BY 4.0

이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

각도 의존 반사와 Fresnel 발광 마스크의 차이를 확인합니다.

실습 절차

반사할 환경이 있는 물 또는 유리 표면을 준비합니다. 카메라 각도만 바꿔 정면과 낮은 시점의 반사를 봅니다. 이후 별도 재질에 Fresnel 노드의 출력을 Emissive에 연결하고 조명을 줄여 비교합니다.

예상 결과와 이유

기본 표면에서는 각도에 따라 환경 반사의 비중이 달라질 것입니다. 발광 마스크는 환경이 어두워져도 스치는 부분을 밝힐 수 있습니다.

결과를 어떻게 해석할까

두 결과가 비슷해 보여도 빛을 만드는 원인이 다릅니다. Fresnel 노드의 마스크를 실제 물질의 반사율 전체와 동일시하지 않습니다.

실제 제작에서의 선택

자연스러운 유리·물은 기본 반사 모델과 거칠기부터 맞추고, 림 발광은 필요한 연출로 명시해 추가합니다.

이해한 뒤 확인할 조건

F0는 정면 입사에서의 반사율입니다. 굴절률과 금속의 파장별 반응 같은 세부는 공식 PBR 자료에서 확장합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[08] Physically Based Materials

[09] Using Fresnel

[10] Overview of Substrate Materials

[31] Real Shading in Unreal Engine 4 · Brian Karis

10 셰이더와 WPO

중심 질문

재질에서 풀을 흔들면 왜 형상과 그림자가 함께 달라질까?

먼저 알아둘 답

WPO는 텍스처를 움직이는 대신 기존 정점의 위치를 바꾸므로, 변형된 형상이 표면과 그림자 계산에 전달됩니다.

왜 알아야 할까

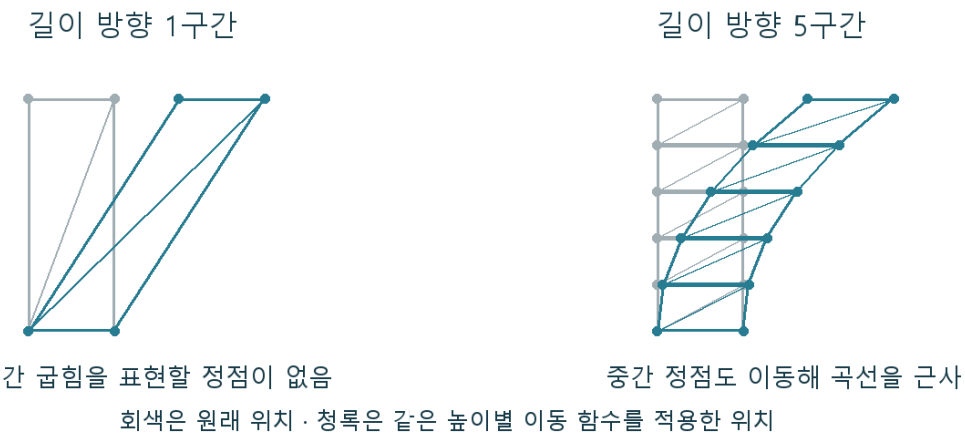
식생과 천의 움직임을 만들 때 정점 배치와 변형 범위가 결과를 어떻게 제한하는지 이해하기 위해 배웁니다.

왜 이런 결과가 생길까

셰이더는 GPU에서 실행하는 계산 프로그램입니다. 머티리얼 그래프는 그 계산을 노드로 작성하는 도구입니다. World Position Offset은 정점 위치에 더할 이동량을 지정합니다. 밑부분은 0, 끝부분은 큰 이동량을 주면 줄기가 휘어 보입니다. 단, 기존 정점 사이의 삼각형은 그대로 연결되므로 정점이 없는 곳에 새로운 굽힘 구간을 만들지는 않습니다.

제작 중에는 이렇게 나타납니다

정점이 적은 긴 풀은 끝만 움직이거나 판 전체가 꺾여 보일 수 있습니다. 텍스처 해상도를 올려도 이런 실루엣 문제는 해결되지 않습니다. 움직인 정점이 그림자에 참여하면 그림자 모양도 바뀝니다.



직접 제작 기하 예시. 정점 사이를 직선으로 잇기 때문에 같은 변위 함수라도 정점 배치에 따라 굽힘 표현이 달라집니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

같은 변위 함수에서 정점의 구간 수가 굽힘에 미치는 영향을 확인합니다.

실습 절차

길이 방향으로 1구간인 카드와 여러 구간인 카드를 만듭니다. 같은 높이 기반 가중치와 같은 바람 함수를 연결합니다. 변형 크기를 고정하고 옆면 실루엣과 바닥 그림자를 비교합니다.

예상 결과와 이유

여러 구간 카드는 중간 정점도 이동해 휘어 보이고, 1구간 카드는 중간 굽힘을 표현하지 못할 것입니다. WPO를 끄면 두 카드가 원래 형상으로 돌아옵니다.

결과를 어떻게 해석할까

외관 비교 뒤 정점 수와 그림자 조건을 기록하고 비용을 따로 봅니다. 움직임이 화면 가장자리에서 사라지면 메시를 감싸는 판정 범위인 Bounds도 확인합니다.

실제 제작에서의 선택

가까이 읽히는 굽힘에 필요한 곳에 정점을 배치합니다. 먼 식생의 변형과 그림자 갱신을 줄일지는 CH22의 원리와 함께 판단합니다.

이해한 뒤 확인할 조건

셰이더 순열은 설정 조합별로 준비하는 프로그램 변형입니다. 실행 중 WPO 비용과 컴파일·쿠킹 준비 비용은 다른 문제이므로 이 실습의 결론에 섞지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[02] Render Dependency Graph

[11] Material Editor UI

[12] Material Inputs

11 Substrate와 물질 적층

중심 질문

페인트 위 투명 코팅은 왜 두 재질의 색을 섞는 것만으로 충분하지 않을까?

먼저 알아둘 답

위아래 물질은 빛이 차례로 통과하는 관계를 가지므로, 단순한 파라미터 평균과 다른 반응을 만듭니다.

왜 알아야 할까

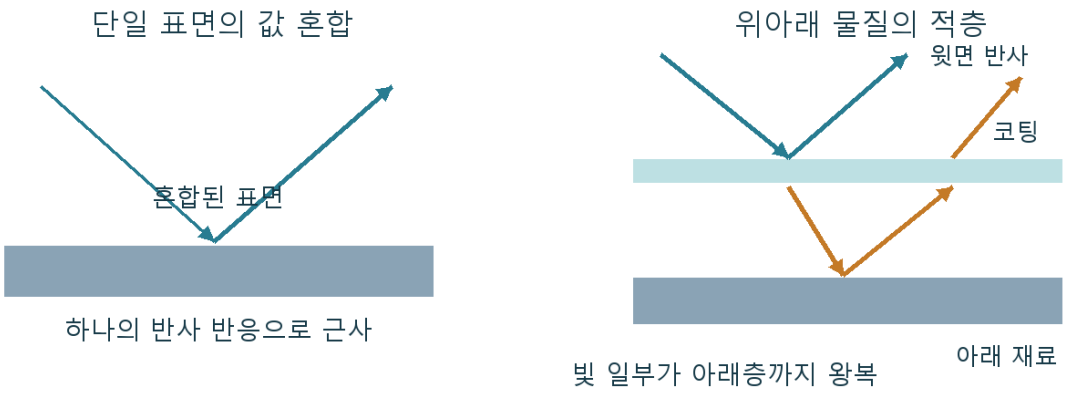
도장·코팅·물기 같은 복합 표면에서 혼합과 적층 중 어떤 구조가 필요한지 선택하기 위해 배웁니다.

왜 이런 결과가 생길까

두 재료를 나란히 섞은 표면에서는 위치마다 서로 다른 재료가 보입니다. 반면 코팅에서는 빛 일부가 윗면에서 반사되고 나머지가 아래층에 도달한 뒤 다시 나옵니다. 아래층의 빛은 윗층의 영향을 받습니다. Substrate는 빛과 물질의 상호작용을 표현하는 Slab과 연산자를 연결해 이런 관계를 구성합니다.

제작 중에는 이렇게 나타납니다

아래 페인트는 거칠고 위 코팅은 매끈한 경우, 위층의 선명한 반사와 아래층의 넓은 반응이 함께 필요할 수 있습니다. 거칠기 하나를 평균하면 그 두 반응을 하나로 바꿔 버립니다.



직접 제작한 빛 경로 개념도 · 실제 추적·단순화 구현을 그대로 묘사한 그림은 아닙니다.

직접 제작 개념도. 한 표면의 파라미터 혼합과, 위아래 물질을 통과하는 빛의 관계를 구분합니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

단일 표면 근사와 위아래 적층의 반응을 여러 각도에서 비교합니다.

실습 절차

같은 구에 아래층의 색·거칠기를 고정합니다. 하나는 그 위에 별도 Slab을 수직 적층하고, 다른 하나는 단일 Slab의 값을 조정한 근사 버전으로 둡니다. 카메라와 광원을 유지하며 넓은 각도로 관찰합니다.

예상 결과와 이유

적층 버전에서는 위층 반사와 아래층 반응을 독립적으로 조절할 여지가 생깁니다. 단일 표면은 한 각도에서 비슷해도 다른 각도에서 두 반응을 동시에 맞추기 어려울 수 있습니다.

결과를 어떻게 해석할까

결과가 비슷하다면 조명과 관찰 각도가 층 차이를 드러내는지 확인합니다. 실제 실시간 결과에는 단순화가 개입할 수 있으므로 사용한 설정도 기록합니다.

실제 제작에서의 선택

여러 각도에서 읽히는 층 반응이 필요하면 적층을 검토합니다. 차이가 거의 읽히지 않는 원거리 표면은 단순한 재질로 근사하고 비용을 비교합니다.

이해한 뒤 확인할 조건

Slab 수만으로 고정 비용을 단정하지 않습니다. 저장 형식·단순화·플랫폼 지원은 버전에 따라 확인할 심화 내용입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[05] Programming with Substrate GBuffer Formats

[06] Decal Materials

[10] Overview of Substrate Materials

12 Translucency와 합성

중심 질문

유리판을 겹치면 왜 불투명 벽과 다른 오류가 생길까?

먼저 알아둘 답

반투명은 앞면 하나만 남겨서는 색을 결정할 수 없고, 뒤의 색과 여러 층을 순서에 맞춰 합쳐야 합니다.

왜 알아야 할까

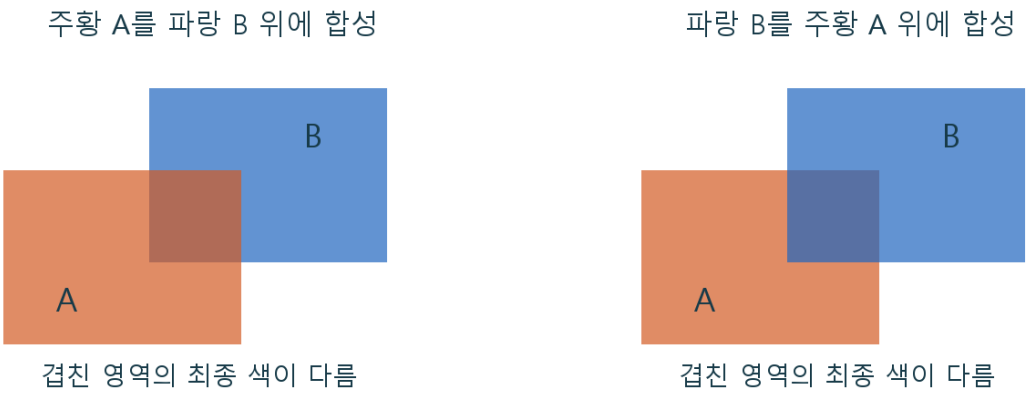
유리·물·연기에서 겹침과 정렬 오류의 원인을 구분하고 레이어 구성을 설계하기 위해 배웁니다.

왜 이런 결과가 생길까

불투명 벽은 앞면이 뒤를 완전히 가리지만 유리는 뒤의 색도 필요합니다. 일반적인 알파 합성은 앞 색의 기여와 뒤 색의 기여를 섞습니다. 이 연산은 순서를 바꾸면 결과가 달라질 수 있습니다. 엔진이 물체 단위로 정렬하는 조건에서는 서로 교차하는 복잡한 표면을 모든 픽셀에서 올바른 순서로 처리하기 어렵습니다.

제작 중에는 이렇게 나타납니다

유리의 이상한 겹침은 Base Color를 조정해 해결할 문제가 아닐 수 있습니다. 굴절은 배경의 다른 위치를 읽거나 추적하는 별도의 문제이며, 먼저 합성 순서를 확인해야 원인을 나눌 수 있습니다.



직접 계산한 알파 합성 개념도 · 같은 두 색과 알파 0.7에서 합성 순서만 변경

직접 제작 합성 예시. 두 색과 알파는 같지만 합성 순서가 달라 겹친 색이 달라집니다. 실제 엔진 캡처가 아닌 기본 알파 연산의 비교입니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

굴절을 끈 두 판으로 합성 순서와 교차 오류를 분리해 관찰합니다.

실습 절차

굴절을 끈 서로 다른 색의 반투명 판 두 개를 둡니다. 판을 떨어뜨린 상태와 교차시킨 상태에서 카메라만 천천히 이동합니다. 불투명 판으로 바꾼 비교도 남깁니다.

예상 결과와 이유

단순히 앞뒤로 떨어진 경우보다 교차한 경우에 겹침 순서가 갑자기 바뀌거나 일부 영역이 어색해질 가능성이 높습니다. 불투명 비교에서는 앞 표면의 깊이가 우선합니다.

결과를 어떻게 해석할까

굴절 없이도 오류가 나면 먼저 정렬·교차 구조를 조사합니다. 굴절을 켠 뒤 생기는 화면 가장자리 누락은 별도 실험으로 다룹니다.

실제 제작에서의 선택

겹친 반투명 층과 교차를 줄이고 필요한 실루엣을 유지하도록 메시를 구성합니다. 큰 화면 점유율과 층 수는 합성 비용도 늘릴 수 있습니다.

이해한 뒤 확인할 조건

모든 반투명 모드가 같은 정렬과 추적을 사용하지는 않습니다. 이 장의 기본 알파 합성을 이해한 뒤 프로젝트의 선택 경로를 확인합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[13] Lit Translucency

[14] Using Refraction

13 SSS와 표면 아래 산란

중심 질문

왁스는 왜 두께와 역광에 따라 색과 밝기가 달라질까?

먼저 알아둘 답

빛이 표면에서 바로 반사되는 대신 내부를 지나며 흩어지고 흡수되기 때문입니다.

왜 알아야 할까

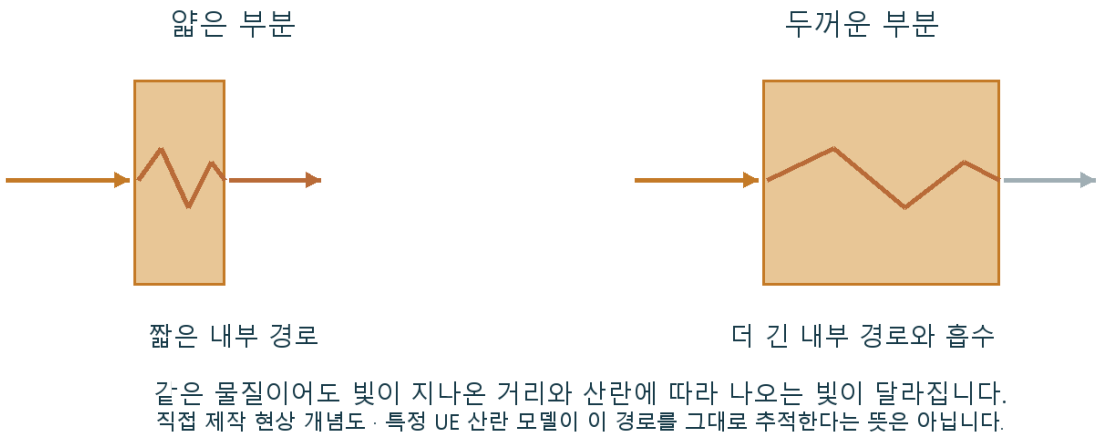
왁스·피부·일부 돌과 식생의 밀도감을 단순 투명도와 구분해 제작하기 위해 배웁니다.

왜 이런 결과가 생길까

Subsurface Scattering은 들어온 빛이 내부에서 산란한 뒤 다른 위치나 방향으로 나오는 현상입니다. 지나온 거리와 물질의 흡수 특성에 따라 나오는 빛의 색과 양이 달라집니다. 얇은 부분과 두꺼운 부분의 반응이 다른 이유입니다. 일반 표면 반사만으로는 이 부드러운 내부 빛 전달을 충분히 표현하기 어렵습니다.

제작 중에는 이렇게 나타납니다

알파를 낮춰 뒤 배경이 비치게 하는 것과 빛이 물질 내부를 거쳐 나오는 것은 다릅니다. 왁스의 역광을 만들려고 투명도만 낮추면 재료가 유리처럼 보일 수 있습니다.



같은 물질이어도 빛이 지나온 거리와 산란에 따라 나오는 빛이 달라집니다.
직접 제작 현상 개념도 · 특정 UE 산란 모델이 이 경로를 그대로 추적한다는 뜻은 아닙니다.

직접 제작 빛 전달 개념도. 얇고 두꺼운 부분에서 달라질 수 있는 내부 경로를 비교합니다. 실제 결과는 재료와 선택한 산란 모델에 달려 있습니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

한 산란 모델에서 얇은 부분과 두꺼운 부분의 역광 반응을 비교합니다.

실습 절차

얇은 돌기와 두꺼운 몸통이 있는 형상에 같은 산란 재질을 적용합니다. 한 가지 SSS 모델을 고정하고 전면광과 역광을 각각 관찰합니다. 일반 불투명 표면 재질과도 비교합니다.

예상 결과와 이유

역광에서 얇은 부분의 빛 전달이 두꺼운 몸통과 다르게 보일 것으로 예상합니다. 일반 표면 재질은 같은 내부 산란 반응을 만들지 못할 것입니다.

결과를 어떻게 해석할까

예상 차이가 없으면 선택 모델이 두께를 어떻게 근사하거나 입력받는지 확인합니다. 모델의 입력 의미를 먼저 확인하지 않고 Opacity 숫자를 다른 SSS 방식에 옮기지 않습니다.

실제 제작에서의 선택

원하는 재료의 두께·스케일·조명 방향을 함께 맞춥니다. 산란색만 붉게 바꿔 모든 밀도감을 해결하려 하지 않습니다.

이해한 뒤 확인할 조건

Subsurface, Subsurface Profile, Substrate는 같은 입력값이 같은 의미라고 가정할 수 없습니다. 내부 경로 그림도 현상의 설명이며 선택한 모델의 실제 추적을 재현한 것은 아닙니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[10] Overview of Substrate Materials

[15] Subsurface Profile Shading Model

14 Draw Call과 제출 작업

중심 질문

소품 하나인데 왜 여러 번 그리는 명령이 생길까?

먼저 알아둘 답

그리기 명령은 Actor 개수보다 메시의 구역·재질·참여 패스에 따라 나뉩니다.

왜 알아야 할까

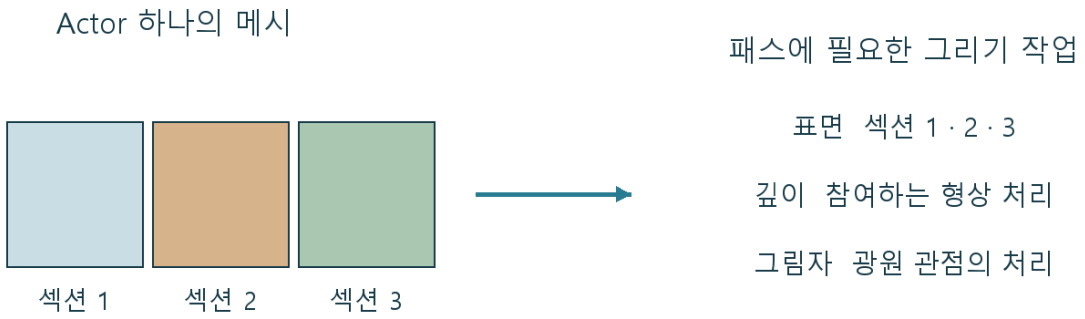
모듈 배경의 재질 슬롯과 합체 단위를 CPU 제출 부담에 근거해 설계하기 위해 배웁니다.

왜 이런 결과가 생길까

Draw Call은 GPU에 정해진 상태로 지오메트리를 그리도록 제출하는 명령입니다. 한 메시가 서로 다른 재질 구역인 섹션으로 나뉘면 별도 그리기 작업이 필요할 수 있습니다. 같은 메시가 깊이·카메라 표면·그림자 작업에 각각 참여하기도 합니다. 그래서 레벨에 배치한 Actor 하나를 명령 하나로 셀 수 없습니다.

제작 중에는 이렇게 나타납니다

오브젝트 수만 줄이려고 메시를 합쳐도 재질 구역이 그대로라면 기대만큼 제출이 줄지 않을 수 있습니다. 반대로 너무 크게 합치면 일부만 보일 때도 전체가 가시 후보로 남을 수 있습니다.



섹션과 패스가 제출 작업을 나눕니다. 자동 묶음과 경로에 따라 실제 명령 수는 달라집니다.

직접 제작 관계도. Actor 수, 메시 섹션, 패스별 그리기 작업을 나누어 보여 줍니다. 명령 수의 고정 공식은 아닙니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

Actor 수를 유지하고 섹션 수에 따른 제출 변화를 확인합니다.

실습 절차

자동 묶음의 영향을 통제한 비교 장면에서 외관과 삼각형이 같은 1섹션 메시와 여러 섹션 메시를 준비합니다. 수량·재질 동작·그림자 조건을 맞추고 stat scenerendering 또는 stat rhi의 관련 그리기 수치를 기록합니다.

예상 결과와 이유

섹션 분리가 패스별 제출을 늘리는 조건이라면 여러 섹션 버전의 그리기 수치가 높아질 것입니다. GPU 시간이 반드시 같은 비율로 늘어나는 것은 아닙니다.

결과를 어떻게 해석할까

stat unit의 Draw는 주로 CPU 렌더 스레드 시간이며 호출 개수가 아닙니다. 호출 수가 줄었는지와 실제 병목 시간이 줄었는지는 따로 기록합니다.

실제 제작에서의 선택

읽히지 않는 슬롯 분할을 줄이고 공유 가능한 재질을 검토합니다. 합체 여부는 제출 이득과 가시성 손실을 함께 비교해 결정합니다.

이해한 뒤 확인할 조건

호환되는 명령을 묶는 배칭과 Nanite의 제출 방식은 결과를 바꿀 수 있습니다. 먼저 사용한 경로를 고정하고 비교합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[03] Introduction to Performance Profiling and Configuration

[16] Mesh Drawing Pipeline

15 Instancing과 반복 배치

중심 질문

같은 기둥 수백 개를 왜 개별 Actor 대신 인스턴스로 관리할까?

먼저 알아둘 답

공통 메시와 재질을 공유하고 배치별 정보만 따로 두면 반복 오브젝트의 관리·제출을 줄일 수 있습니다.

왜 알아야 할까

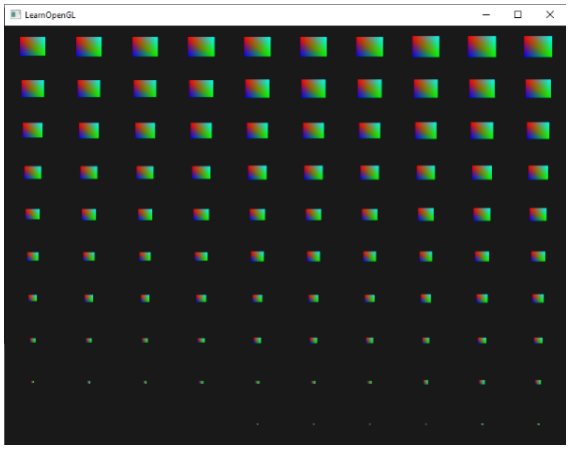
울타리·가로수·군집 소품을 반복 배치할 때 공유할 정보와 개별 정보를 구분하기 위해 배웁니다.

왜 이런 결과가 생길까

같은 기둥은 형상이 같고 위치·회전·크기만 다릅니다. 이런 변환 정보를 Transform이라고 합니다. 인스턴싱은 공통 데이터를 재사용하며 각 배치의 Transform을 따로 보관합니다. UE의 ISM은 Instanced Static Mesh Component로, 여러 배치를 하나의 컴포넌트에서 관리합니다. 반복 데이터를 다루는 준비 작업을 묶을 수 있습니다.

제작 중에는 이렇게 나타납니다

인스턴싱은 화면에 보이는 200개의 기둥을 한 개처럼 한 번만 셰이딩한다는 뜻이 아닙니다. 각 위치의 삼각형·픽셀·그림자는 여전히 처리해야 합니다.



외부 렌더 이미지 · 공통 사각형을 인스턴싱하면서 배치별 위치와 크기를 달리한 예입니다.

같은 기본 형상을 공유해도 복사본마다 변환 정보가 다릅니다. OpenGL 학습 예시이며 UE ISM의 호출 수나 속도를 측정한 자료는 아닙니다. 외관을 유지하며 제출 비용이 줄었는지는 목표 장면에서 검증합니다.

© Joey de Vries · LearnOpenGL · @JoeyDeVriez · 출처 [52]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · CC BY 4.0

이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

동일 반복 배치를 개별 Actor와 ISM으로 관리할 때의 차이를 확인합니다.

실습 절차

같은 메시·재질의 기둥 200개를 개별 Actor로 둔 버전과 ISM으로 둔 버전을 준비합니다.
배치·충돌·그림자를 맞추고 같은 카메라에서 Actor·컴포넌트 구성과 CPU·GPU 시간을 비교합니다.

예상 결과와 이유

ISM에서는 독립적으로 관리할 Actor·컴포넌트 수가 줄 것입니다. CPU 관리나 제출이 문제였던 장면이라면 그 구간의 개선을 기대할 수 있습니다.

결과를 어떻게 해석할까

GPU 픽셀 비용이 지배하면 전체 프레임 변화는 작을 수 있습니다. 엔진의 자동 인스턴싱 때문에 제출 차이가 작아도 데이터 관리 방식의 차이는 남습니다.

실제 제작에서의 선택

공통 메시를 반복하는 곳에 인스턴싱을 검토하고, 배치별 색은 공유 재질의 Per Instance Custom Data 같은 입력으로 전달합니다.

이해한 뒤 확인할 조건

HISM은 공간 계층을 사용하는 확장입니다. 잦은 위치 갱신과 컬링·LOD·Nanite 조건에 따라 선택이 달라지므로 별도 성능 과제로 다룹니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[16] Mesh Drawing Pipeline

[17] Instanced Static Mesh Component

16 LOD와 화면 크기

중심 질문

먼 난간의 나사는 왜 생략해도 되지만 윤곽은 남겨야 할까?

먼저 알아둘 답

화면에서 읽히지 않는 디테일은 줄이고, 적은 픽셀에서도 보이는 형태 정보는 유지해야 합니다.

왜 알아야 할까

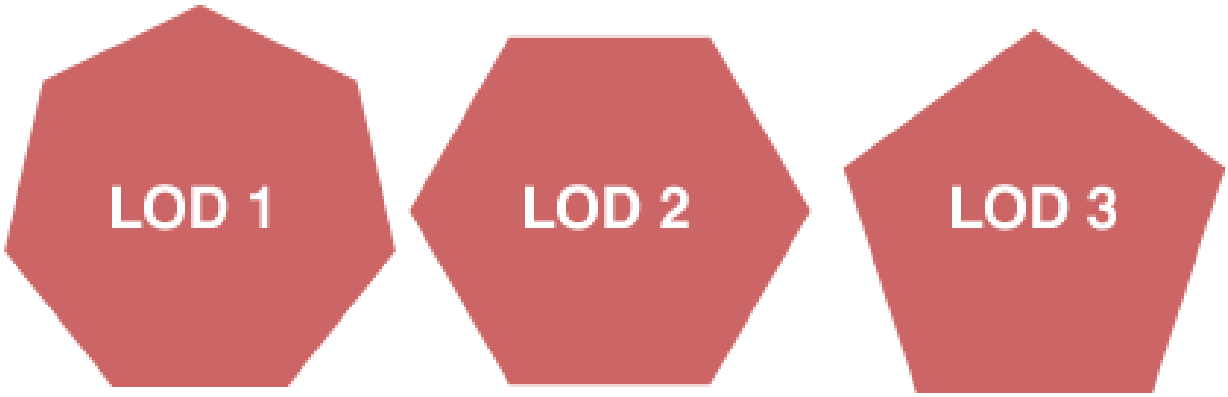
에셋의 디테일을 플레이 거리와 출력 크기에 맞게 배분하기 위해 배웁니다.

왜 이런 결과가 생길까

LOD는 필요한 세부 수준에 따라 표현을 바꾸는 방식입니다. 전통 Static Mesh LOD는 미리 만든 대체 메시를 선택합니다. 멀어질수록 같은 형상이 차지하는 픽셀이 줄어 작은 나사는 보이지 않지만, 기둥의 굵기나 상단 실루엣은 여전히 외관을 좌우합니다. 따라서 원본의 모든 부분을 같은 비율로 줄이는 것이 좋은 단순화는 아닙니다.

제작 중에는 이렇게 나타납니다

UE는 화면 크기를 기준으로 LOD를 선택할 수 있습니다. 거리뿐 아니라 메시 크기와 카메라 시야각인 FOV도 화면 점유율을 바꾸므로 같은 거리에서 항상 같은 LOD가 나오는 것은 아닙니다.



외부 비교 그림 · 비슷한 실루엣을 더 적은 변으로 근사한 세 가지 LOD입니다.

오른쪽으로 갈수록 위쪽 윤곽이 각지게 바뀝니다. 원본의 LOD 1·2·3 표기는 해당 그림의 번호이며 UE에서 흔히 시작하는 LOD0 번호와 구별합니다.

Sirotk · Wikimedia Commons · 출처 [42]

원본 전체 유지 · 비율 유지 · CC BY-SA 3.0

이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

카메라 이동에서 LOD가 잃는 디테일과 남겨야 할 윤곽을 판정합니다.

실습 절차

난간의 원본과 나사·작은 베벨을 줄인 LOD를 준비합니다. 같은 구도에서 강제 LOD로 외관을 비교한 뒤, Screen Size 전환을 켜고 카메라를 천천히 이동합니다.

예상 결과와 이유

먼 구도에서는 작은 부품의 차이가 줄어들지만 실루엣을 많이 바꾸면 전환 순간 형태가 튈 것입니다. 가까운 구도에서는 단순화 손실이 더 잘 보일 것입니다.

결과를 어떻게 해석할까

정지 비교에서 비슷해도 이동 중 전환과 그림자 변화를 확인합니다. 줄인 삼각형 수가 아니라 잃은 시각 정보가 허용되는지를 먼저 판단합니다.

실제 제작에서의 선택

실루엣·큰 간격·그림자 형태를 보존하고 작은 면부터 줄입니다. 이후 GPU 시간이 줄었는지 확인해 실제 병목과 연결합니다.

이해한 뒤 확인할 조건

픽셀 재질이 병목이면 삼각형 감소만으로 전체 이득이 작을 수 있습니다. 디더 전환과 재질 단순화는 다음 단계의 선택입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[17] Instanced Static Mesh Component

[18] Optimizing LOD Screen Size Per Platform

17 Nanite와 가상화 지오메트리

중심 질문

Nanite는 원본의 수많은 삼각형을 매번 모두 그릴까?

먼저 알아둘 답

Nanite는 가시성과 화면 오차를 기준으로 필요한 삼각형 묶음과 세부 수준을 선택합니다.

왜 알아야 할까

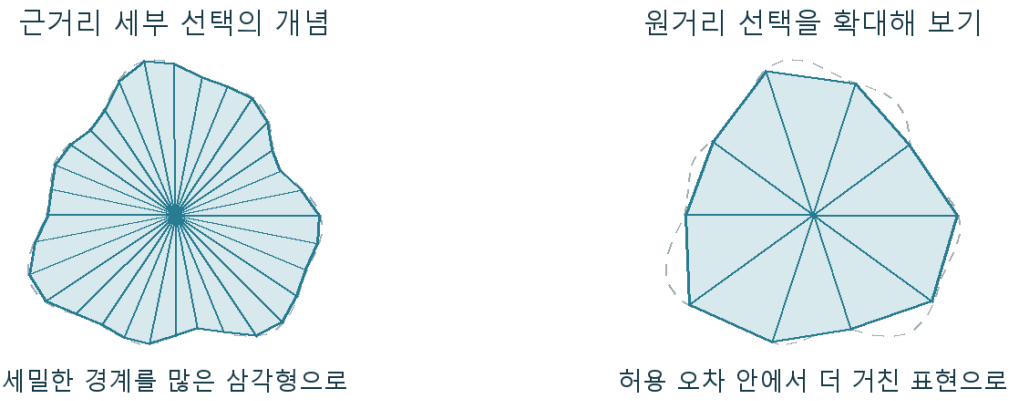
고밀도 배경 에셋에 디테일을 사용할 수 있는 이유와 남아 있는 비용을 이해하기 위해 배웁니다.

왜 이런 결과가 생길까

Nanite는 메시를 작은 삼각형 묶음인 클러스터와 그 계층으로 구성합니다. 가까운 부분에는 세밀한 표현을, 작게 보이는 부분에는 허용 오차 안의 거친 표현을 선택합니다. 필요한 데이터는 스트리밍으로 공급합니다. 원본 삼각형 수가 그대로 매 프레임의 표시 삼각형 수가 되지 않는 이유입니다.

제작 중에는 이렇게 나타납니다

모델링 예산을 원본 폴리곤 하나로만 제한하던 판단은 달라질 수 있습니다. 하지만 같은 화면을 겹쳐 덮는 표면과 재질·조명·그림자 계산까지 사라지는 것은 아닙니다.



직접 제작 설명도 · 실제 Nanite 클러스터 분할이나 측정 개수를 재현한 그림은 아닙니다.

직접 제작 세부 선택 개념도. 원거리 결과도 비교를 위해 같은 크기로 확대했습니다. 줄인 형상과 원래 경계의 차이를 봅니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

원본을 바꾸지 않고 화면 크기에 따라 선택되는 Nanite 세부를 관찰합니다.

실습 절차

Nanite 바위 하나를 고정하고 카메라를 가까이에서 멀리 이동합니다. Nanite Visualization의 Triangles와 Clusters를 각각 관찰합니다. 원본 메시를 수정하지 않은 채 선택된 세부가 어떻게 달라지는지 기록합니다.

예상 결과와 이유

거리에 따라 선택되는 삼각형과 클러스터가 달라지면서 최종 실루엣은 가능한 범위에서 유지될 것입니다. 표시색은 재질색이 아니라 선택 단위를 구분하는 진단입니다.

결과를 어떻게 해석할까

원본 개수와 화면에 선택된 정보를 구분합니다. 자동 선택이 외관에 충분한지는 실제 출력 크기의 실루엣·구멍·그림자로 판단합니다.

실제 제작에서의 선택

고밀도 바위와 건축에서는 원본 디테일을 활용하되, 얇은 잎·겹침·변형은 별도 적합성 검증을 합니다.

이해한 뒤 확인할 조건

Preserve Area의 잎 면적 보존, 폴백 메시와 플랫폼 지원은 적용 대상에 따라 추가 확인합니다. 본문의 개념도는 선택 원리를 설명하며 실제 클러스터 수나 성능을 재현하지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[17] Instanced Static Mesh Component

[19] Nanite Virtualized Geometry

18 Culling과 가시성

중심 질문

건물을 한 덩어리로 합치면 왜 벽 뒤 부분까지 처리할 수 있을까?

먼저 알아둘 답

가시성은 흔히 물체를 감싸는 범위로 판정하므로, 큰 묶음 일부가 보이면 나머지도 후보에 남을 수 있습니다.

왜 알아야 할까

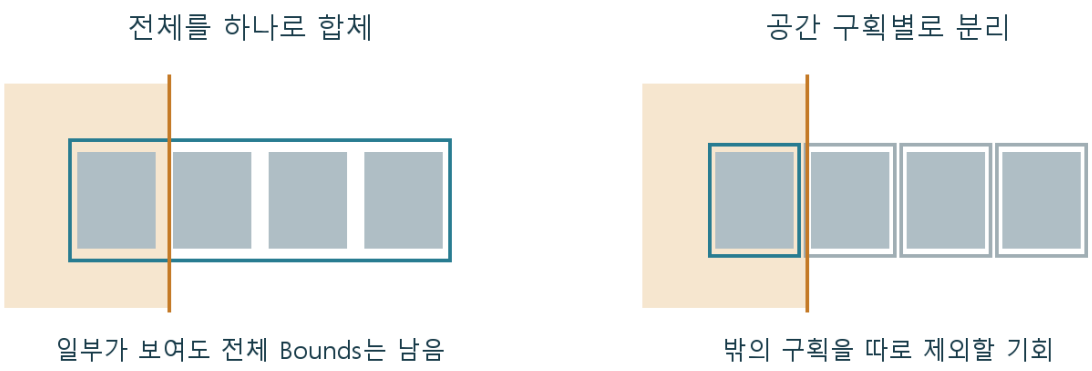
건물·실내 모듈의 분할 단위를 제출 비용과 가림 효율 사이에서 결정하기 위해 배웁니다.

왜 이런 결과가 생길까

컬링은 최종 화면에 불필요한 렌더링 후보를 제외하는 과정입니다. 카메라의 시야 공간인 프러스텀 밖을 제외하는 판정과 다른 표면 뒤에 가려졌는지 보는 판정은 다릅니다. 엔진은 모든 삼각형 대신 물체를 감싸는 Bounds로 후보를 검사할 수 있습니다. 한 건물 전체의 Bounds가 보이면 그 내부의 구역을 따로 제외하기 어려워질 수 있습니다.

제작 중에는 이렇게 나타납니다

메시 합체는 제출 수를 줄일 수 있지만 공간별 제외 기회를 줄일 수 있습니다. 작은 조각으로 무조건 나누면 반대로 오브젝트 관리와 검사 부담이 늘어납니다.



주황 영역은 카메라 범위 안 · 사각 테두리는 Bounds · 같은 건물을 다른 단위로 판정
직접 제작 가시성 개념도 · 실제 원근 시야나 엔진의 모든 컬링 단계를 재현한 그림은 아닙니다.

직접 제작 Bounds 비교. 시야에 일부가 들어온 큰 묶음과, 개별 판정이 가능한 작은 구획의 차이입니다. 가려짐 검사 전의 기본 후보 판정으로 단순화했습니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

합체·구획 버전에서 Bounds와 가림 후보가 어떻게 다른지 확인합니다.

실습 절차

전통 메시로 된 꺾인 복도에서 같은 건물을 통합 버전과 방·구획별 버전으로 준비합니다. Bounds를 표시하고 모서리 앞뒤에서 가시 후보와 관련 렌더링 수치를 비교합니다.

예상 결과와 이유

구획 버전은 완전히 가린 부분을 별도 후보로 제외할 기회가 더 많을 것입니다. 통합 버전은 작은 부분만 보여도 큰 범위가 남을 수 있습니다.

결과를 어떻게 해석할까

메인 화면에 안 보인다고 그림자 작업까지 모두 사라진 것은 아닙니다. 카메라 가시성과 조명 관점의 기여를 구분한 뒤 전체 비용을 비교합니다.

실제 제작에서의 선택

작업 편의만으로 합체하지 말고, 실제 카메라에서 가려지는 공간 단위를 분할 기준에 포함합니다.

이해한 뒤 확인할 조건

Nanite의 클러스터 컬링과 일반 Actor Bounds 판정은 같은 단위가 아닙니다. 이 실습에서는 경로를 고정합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[17] Instanced Static Mesh Component

[19] Nanite Virtualized Geometry

[30] Visibility and Occlusion Culling

19 Mipmap과 샘플링

중심 질문

먼 바닥의 잔무늬는 왜 고해상도 원본만 쓰면 더 흔들릴까?

먼저 알아둘 답

작은 화면 픽셀이 많은 텍셀을 대표해야 하므로, 적절히 축소한 정보를 읽어야 안정적입니다.

왜 알아야 할까

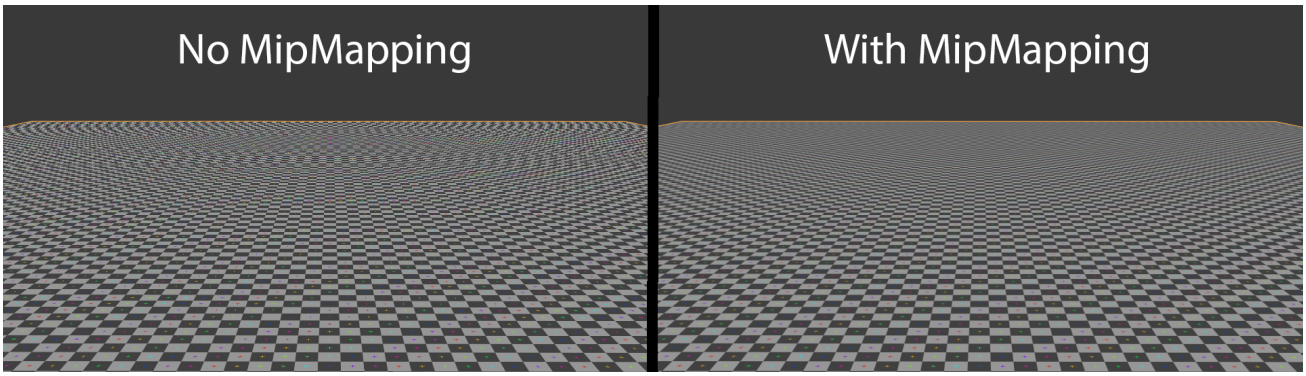
원거리 패턴과 미세 노멀의 깜빡임을 텍스처 해상도 부족과 구분하기 위해 배웁니다.

왜 이런 결과가 생길까

텍스처의 한 데이터 칸을 텍셀이라고 합니다. 멀리 있는 바닥에서는 화면 한 픽셀이 많은 텍셀을 덮습니다. 원본의 한 점만 읽으면 카메라의 작은 이동에도 검은 줄과 흰 줄을 번갈아 고를 수 있습니다. 밉맵은 원본을 단계적으로 축소해 넓은 영역을 대표하는 값을 준비합니다. 필터링은 읽은 값들을 섞어 표본의 불연속을 줄입니다.

제작 중에는 이렇게 나타납니다

원본을 더 선명하게 만드는 일과 멀리서 안정적으로 보이게 하는 일은 다릅니다. 너무 촘촘한 패턴은 원본이 선명해도 화면에서 충분히 표현할 수 없습니다.



외부 실제 렌더 비교 · 왼쪽 No MipMapping, 오른쪽 With MipMapping입니다.

수평선에 가까운 먼 바닥에서 왼쪽의 물결무늬와 오른쪽의 안정된 패턴을 비교합니다. 원본은 Blender 렌더이며 필터링도 관여할 수 있으므로 UE에서 밉만 바꾼 통제 실험으로 해석하지 않습니다.

BillyBob CornCob · Wikimedia Commons · 출처 [43]

원본 전체 유지 · 비율 유지 · CC0 1.0

이용 조건

예상한 결과를 장면에서 확인하기

이 예제의 목표

같은 이동에서 밉의 선명도 손실과 시간적 안정성을 함께 관찰합니다.

실습 절차

잔무늬 바닥의 일반 밉 버전과 NoMipmaps 버전을 준비합니다. 원하는 밉이 상주한 조건에서 같은 낮은 시선으로 천천히 이동하며 원거리 영역을 녹화합니다.

예상 결과와 이유

밉이 없는 버전은 먼 패턴의 모아레와 깜빡임이 커지고, 일반 밉은 일부 선명도를 줄이면서 더 안정될 것으로 예상합니다.

결과를 어떻게 해석할까

정지 화면의 선명도만 고르지 않습니다. 비스듬한 바닥에서는 이방성 필터링도 관여하므로 그 설정을 고정한 뒤 밉 차이를 봅니다.

실제 제작에서의 선택

플레이 거리에서 읽힐 패턴 크기와 노멀 강도를 먼저 맞추고 밉을 사용합니다. 흐림이 과하면 생성·필터·상주 상태를 순서대로 조사합니다.

이해한 뒤 확인할 조건

밉 체인의 데이터 증가와 캐시·대역폭 이득은 성능의 확장 주제입니다. 밉 선택과 실제 메모리 상주는 CH20에서 구분합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[20] Texture Asset Editor

[21] Texture Streaming Overview

20 Texture Streaming

중심 질문

텍스처 원본은 선명한데 게임에서는 왜 잠시 흐리게 보일까?

먼저 알아둘 답

화면에 필요한 밉과 현재 메모리에 준비된 밉이 다를 수 있기 때문입니다.

왜 알아야 할까

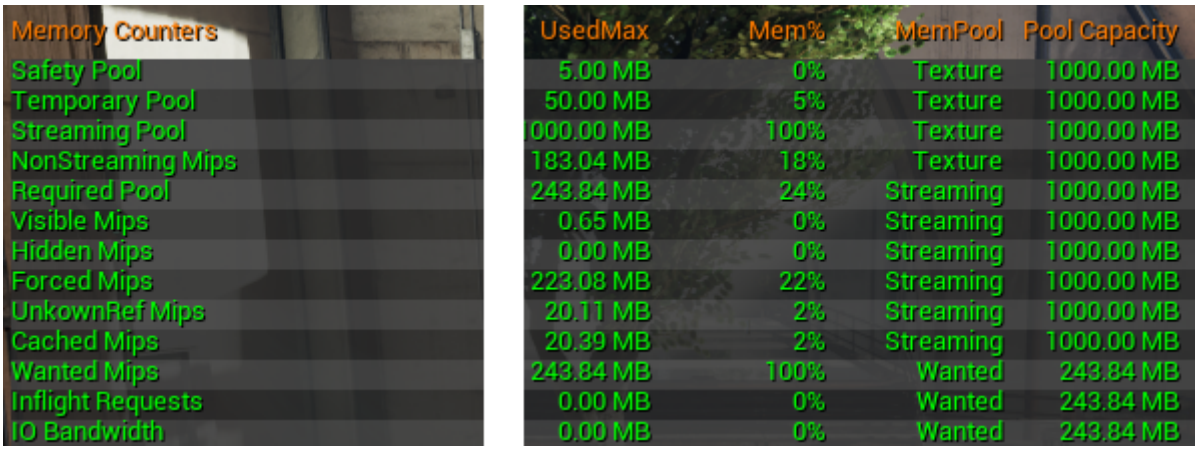
텍스처 자체의 해상도와 메모리·로딩 문제를 구분해 불필요한 고해상도 제작을 피하기 위해 배웁니다.

왜 이런 결과가 생길까

스트리머는 화면 크기와 거리 등을 바탕으로 필요한 텍스처 세부 수준을 추정합니다. 이후 메모리 예산과 우선순위 안에서 데이터를 불러오거나 내보냅니다. 디스크에서 메모리로 옮기는 데는 시간이 걸리므로 수요가 생긴 즉시 최고 밉이 준비된다고 보장할 수 없습니다. 수요·예산·실제 상주량은 서로 다른 값입니다.

제작 중에는 이렇게 나타납니다

빠른 카메라 이동 후 잠시 흐렸다가 선명해지는 현상은 원본이 작아서 생긴 흐림과 다릅니다. 풀 경고가 있으면 예산 압박도 조사해야 하지만 풀 크기만 늘리면 다른 메모리 여유가 줄 수 있습니다.



Memory Counters	UsedMax	Mem%	MemPool	Pool Capacity
Safety Pool	5.00 MB	0%	Texture	1000.00 MB
Temporary Pool	50.00 MB	5%	Texture	1000.00 MB
Streaming Pool	000.00 MB	100%	Texture	1000.00 MB
NonStreaming Mips	183.04 MB	18%	Texture	1000.00 MB
Required Pool	243.84 MB	24%	Streaming	1000.00 MB
Visible Mips	0.65 MB	0%	Streaming	1000.00 MB
Hidden Mips	0.00 MB	0%	Streaming	1000.00 MB
Forced Mips	223.08 MB	22%	Streaming	1000.00 MB
UnkownRef Mips	20.11 MB	2%	Streaming	1000.00 MB
Cached Mips	20.39 MB	2%	Streaming	1000.00 MB
Wanted Mips	243.84 MB	100%	Wanted	243.84 MB
Inflight Requests	0.00 MB	0%	Wanted	243.84 MB
IO Bandwidth	0.00 MB	0%	Wanted	243.84 MB

Epic 공식 통계 화면 · stat streaming의 Memory Counters를 발췌하고 항목과 수치 사이의 여백을 줄였습니다.

Memory Counters의 Pool·Required·Visible 등 서로 다른 항목을 구분합니다. 화면의 숫자는 원문 예시이며 권장 예산이 아닙니다. UE 5.5 자료이므로 항목 배치는 현재 빌드에서 확인합니다.

© Epic Games · 공식 문서 · 출처 [54]
원본 통계 표 발췌 · 가운데 여백만 축소 · 값과 항목 유지 · 원저작권과 원문 출처 표시는 유지합니다

예상한 결과를 장면에서 확인하기

이 예제의 목표

첫 진입과 재방문에서 실제 밉 준비 상태와 선명해지는 지연을 대조합니다.

실습 절차

고정된 이동 경로를 처음 지나갈 때와 곧바로 재방문할 때 비교합니다. stat streaming에서 필요한 밉, 메모리 예산, 상주·요청 상태를 기록하고 같은 위치의 팝인을 녹화합니다.

예상 결과와 이유

처음 데이터가 준비되지 않은 조건에서는 선명해지는 지연이 더 클 수 있습니다. 재방문 때 필요한 밉이 남아 있다면 더 빠르게 선명해질 것입니다.

결과를 어떻게 해석할까

두 번 모두 계속 흐리면 예산·수요 추정·최대 해상도를 봅니다. 재방문 개선만으로 원인을 단정하지 말고 실제 밉의 준비 상태와 대조합니다.

실제 제작에서의 선택

가까이 읽히는 소품에 해상도를 배분하고, 먼 소품군의 요구량을 낮춰 예산과 외관을 함께 맞춥니다.

이해한 뒤 확인할 조건

일반 Texture Streaming은 Virtual Texturing이나 월드 Actor 스트리밍과 같은 기능이 아닙니다. 표의 외부 숫자는 권장 예산이 아닌 항목 설명용입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[20] Texture Asset Editor

[21] Texture Streaming Overview

[22] Texture Streaming Metrics

21 Lumen과 간접광

중심 질문

직접 빛이 닿지 않는 흰 벽은 왜 주변 벽의 색을 띨까?

먼저 알아둘 답

빛은 첫 표면에서 끝나지 않고 다른 표면으로 전달되며, Lumen은 이 간접광을 실시간으로 근사합니다.

왜 알아야 할까

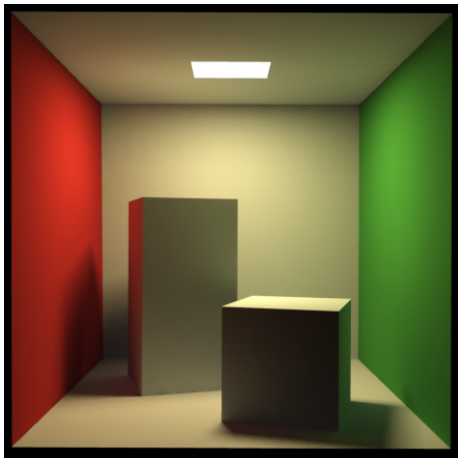
실내의 밝기·색 번짐을 재질색과 구분하고, 라이트를 추가하기 전에 빛 전달을 진단하기 위해 배웁니다.

왜 이런 결과가 생길까

광원에서 바로 도달한 빛은 직접광입니다. 붉은 벽에 닿은 빛 일부가 다시 방 안으로 퍼져 흰 벽에 도달하면 붉은 간접광이 생깁니다. Lumen은 화면 정보와 별도 장면 표현을 이용해 이런 빛의 전달을 추정합니다. 화면에 보이는 정보만 쓰면 화면 밖 물체를 놓칠 수 있으므로 보완할 장면 정보가 필요합니다.

제작 중에는 이렇게 나타납니다

흰 벽이 붉게 보인다고 Base Color를 푸르게 보정하면 다른 조명에서 재질이 틀어질 수 있습니다. 먼저 표면의 고유 색과 들어오는 간접광을 구분해야 합니다.



외부 렌더 참고. 왼쪽 빨강·오른쪽 초록 벽이 주변 중성 표면의 색에 영향을 줍니다. 경로 추적 Cornell box로 간접광 현상을 보여 주며, Lumen 실행 결과는 아닙니다.

Trevorgoodchild · Public domain · 출처 [36]

예상한 결과를 장면에서 확인하기

이 예제의 목표

색 벽만 바꿔 흰 벽의 재질색과 간접광을 구분합니다.

실습 절차

한쪽 벽만 색이 있는 작은 방과 흰 수신 벽을 준비합니다. Lumen GI와 노출·직접광을 고정합니다. 색 벽의 Base Color만 빨강에서 중성 회색으로 바꾸고 수신 벽의 Base Color와 Lit를 비교합니다.

예상 결과와 이유

수신 벽의 Base Color는 유지되고 Lit의 색 번짐은 달라질 것입니다. 직접광은 같아도 주변 재료가 간접 조명을 바꾼다는 예상입니다.

결과를 어떻게 해석할까

변화가 없으면 GI 설정과 장면 표현을 먼저 확인합니다. Surface Cache는 추적한 표면의 재질·조명을 조회하는 저장 체계입니다. 누락을 조사할 때 진단하지만 이 장의 첫 결론은 색 번짐의 원인입니다.

실제 제작에서의 선택

수신 벽의 텍스처를 바로 보정하지 말고 주변 표면·간접광·노출을 조사합니다. 표현 누락은 Lumen Scene과 Surface Cache를 다음 진단 대상으로 삼습니다.

이해한 뒤 확인할 조건

반사에서 Surface Cache와 Hit Lighting을 비교하는 것은 별도 문제입니다. 반사 모드나 추적 옵션을 이 간접광 실험과 동시에 바꾸지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[23] Lumen Technical Details

[24] Lumen Performance Guide

22 Virtual Shadow Maps

중심 질문

바람 부는 풀밭은 왜 카메라가 멈춰도 그림자 작업이 생길까?

먼저 알아둘 답

캐시한 그림자는 형상이 변하면 다시 계산해야 하므로, 정점 변형은 그림자 재사용을 줄일 수 있습니다.

왜 알아야 할까

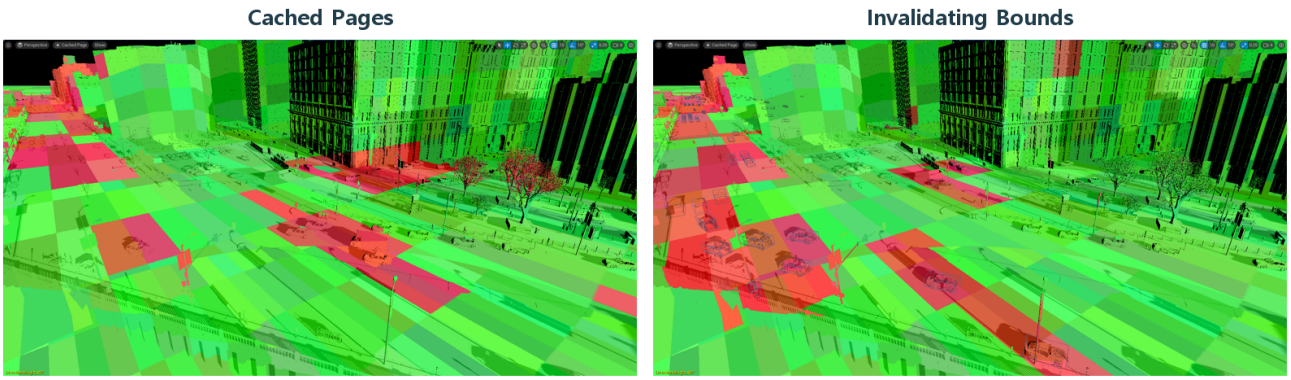
식생의 움직임과 그림자 품질을 유지하면서 먼 배경의 갱신 범위를 설계하기 위해 배웁니다.

왜 이런 결과가 생길까

VSM은 높은 해상도의 그림자를 필요한 작은 페이지 단위로 관리합니다. 장면과 광원이 변하지 않으면 준비한 페이지를 다시 사용할 수 있습니다. WPO로 잎의 위치가 바뀌면 기존 그림자는 낡은 정보가 됩니다. 영향을 받는 영역의 페이지를 무효화하고 새로 그려야 움직이는 그림자가 맞습니다.

제작 중에는 이렇게 나타납니다

카메라가 정지해도 움직이는 식생 때문에 갱신은 계속될 수 있습니다. 그림자 선명도만 낮추기 전에 무엇이 캐시를 깨뜨리는지 보면 더 적절한 수정 대상을 찾을 수 있습니다.



Epic 공식 진단 비교 · 왼쪽은 캐시 페이지, 오른쪽은 무효화를 유발하는 Bounds도 표시한 화면입니다.

이 시각화에서 초록은 재사용, 빨강은 새로 필요하거나 무효화된 페이지입니다. 오른쪽 경계가 어떤 영역에 영향을 주는지 봅니다. UE 5.4 문서의 표시 예이며 비용 측정은 진단 모드를 끈 뒤 수행합니다.

© Epic Games · 공식 문서 · 출처 [56]
동일 범위의 원본을 나란히 배치 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

예상한 결과를 장면에서 확인하기

이 예제의 목표

정지 카메라에서 WPO만 바꿔 그림자 페이지 갱신의 원인을 확인합니다.

실습 절차

카메라와 광원을 고정한 식생 장면에서 바람 WPO만 켜고 끕니다. VSM의 Cached Page와 무효화 Bounds 시각화를 비교합니다. 이후 진단을 끄고 그림자 생성 시간과 전체 GPU 시간을 기록합니다.

예상 결과와 이유

바람이 켜진 상태는 해당 식생 주변에 지속적인 페이지 갱신이 생길 수 있습니다. 바람을 끄면 정적 영역의 재사용 비중이 높아질 것으로 예상합니다.

결과를 어떻게 해석할까

새로 보인 영역 때문에 만든 페이지와 변형 때문에 다시 만든 페이지를 구분합니다. 시각화 변화가 실제 실행 시간에 얼마나 영향을 줬는지는 별도로 측정합니다.

실제 제작에서의 선택

읽히지 않는 원거리 변형을 줄이거나 Bounds와 그림자 참여를 조정합니다. 움직임을 남긴 채 무효화만 막으면 낡은 그림자가 생길 수 있습니다.

이해한 뒤 확인할 조건

페이지 색과 명령은 버전별 표시를 확인합니다. 메모리 예산·투영 비용은 갱신 원리를 이해한 뒤 확장할 항목입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[19] Nanite Virtualized Geometry

[25] Virtual Shadow Maps

23 TAA와 프레임 히스토리

중심 질문

철망의 깜빡임을 줄였는데 왜 움직일 때 잔상이 생길까?

먼저 알아둘 답

이전 프레임을 재사용하면 부족한 샘플을 보완할 수 있지만, 잘못 대응한 과거 정보는 흔적으로 남을 수 있습니다.

왜 알아야 할까

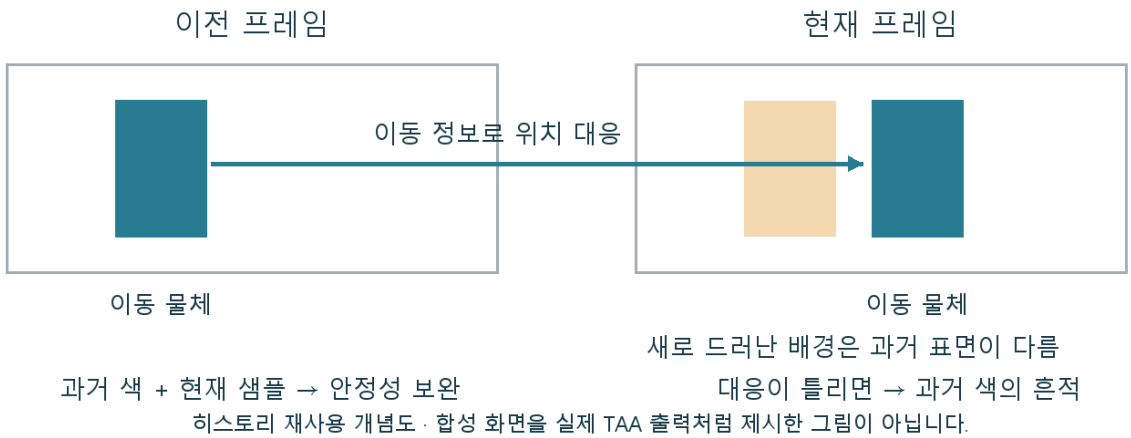
얇은 선·식생·이동 재질의 안정성과 잔상 사이에서 제작 원인을 진단하기 위해 배웁니다.

왜 이런 결과가 생길까

화면 픽셀은 연속된 형상을 제한된 위치에서 검사합니다. 얇은 선이 검사 위치 사이로 오가면 프레임마다 보였다 사라질 수 있습니다. TAA는 프레임마다 조금 다른 위치의 샘플을 얻고 과거 결과를 현재 위치에 대응시켜 누적합니다. 이 과거 결과가 히스토리입니다. 카메라와 물체의 이동 정보가 대응을 도와줍니다.

제작 중에는 이렇게 나타납니다

여러 프레임을 합치면 경계가 안정되지만, 새로 드러난 배경에는 신뢰할 과거 정보가 없고 잘못된 이동 정보는 다른 표면의 색을 가져올 수 있습니다. 깜빡임과 잔상을 함께 봐야 하는 이유입니다.



히스토리 재사용 개념도 · 합성 화면을 실제 TAA 출력처럼 제시한 그림이 아닙니다.

직접 제작 시간 흐름도. 물체의 이전 위치를 현재 위치에 대응시키는 과정과 새로 드러난 배경의 문제를 보여 줍니다.

예상한 결과를 장면에서 확인하기

이 예제의 목표

철망의 정지·이동·가림 해제를 따로 기록해 누적의 이점과 흔적을 봅니다.

실습 절차

내부 해상도 100%에서 TAA와 AA 없음의 철망을 같은 속도로 이동 촬영합니다. 정지한 뒤, 등속 이동, 전경 물체가 지나가 배경이 드러나는 순간을 각각 기록합니다.

예상 결과와 이유

TAA는 정지·완만한 이동에서 얇은 경계를 더 안정시킬 수 있습니다. 새로 드러나는 영역이나 빠른 움직임에서는 재구성 흔적이 더 잘 보일 수 있습니다.

결과를 어떻게 해석할까

정지 이미지 한 장으로 시간적 품질을 판정하지 않습니다. 변형 식생에서만 흔적이 심하면 WPO와 Velocity 출력도 조사합니다. Velocity는 화면상의 이동을 나타내는 정보입니다.

실제 제작에서의 선택

패턴을 과도하게 가늘게 만들지 않고, 미세 노멀과 변형의 이동 정보도 함께 관리합니다. AA 품질 스위치 하나만으로 모든 잔상을 해결하려 하지 않습니다.

이해한 뒤 확인할 조건

히스토리 거부와 필터 세부는 구현마다 다릅니다. 이 장은 TAA의 원리이며 TSR의 입력 해상도 문제는 다음 장에서 다룹니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

24 TSR과 시간적 재구성

중심 질문

낮은 내부 해상도로 그려도 어떻게 높은 출력 해상도를 만들까?

먼저 알아둘 답

TSR은 현재의 저해상도 정보와 과거 프레임을 이용해 출력 픽셀을 재구성합니다.

왜 알아야 할까

목표 출력 화질을 유지하면서 내부 렌더링 해상도에 쓸 GPU 예산을 정하기 위해 배웁니다.

왜 이런 결과가 생길까

내부 해상도는 장면을 계산할 픽셀 수이고 출력 해상도는 화면에 표시할 픽셀 수입니다. 내부 해상도를 낮추면 많은 화면 단위 작업을 줄일 수 있습니다. TSR은 단순히 한 장을 확대하는 대신 이동 정보와 히스토리를 이용해 높은 출력 해상도를 구성합니다. 줄인 입력 정보의 한계를 시간적 정보로 보완하는 방식입니다.

제작 중에는 이렇게 나타납니다

출력 크기가 같아도 처음부터 모든 세부를 같은 수의 샘플로 계산한 것은 아닙니다. 새로 드러난 글자·얇은 선·빠른 반사는 과거 정보를 활용하기 어려워 품질 차이를 드러낼 수 있습니다.

Native 4K



1080p → 4K TSR



Epic 공식 TSR 비교 · 왼쪽 네이티브 4K, 오른쪽 1080p에서 4K로 재구성한 결과의 같은 구역을 발췌했습니다.

출력 크기가 같아도 들어온 정보량은 다릅니다. 글자·창틀 같은 세부를 비교하되 정지된 한 장이 이동 품질을 보장하지는 않습니다. UE 5.5 원문 장면의 성능 수치를 이 프로젝트의 예상 이득으로 사용하지 않습니다.

© Epic Games · 공식 문서 · 출처 [58]
원본 일부 발췌·확대 · 색상 유지 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

예상한 결과를 장면에서 확인하기

이 예제의 목표

출력 크기를 유지하고 입력 해상도에 따른 총 시간과 이동 품질을 비교합니다.

실습 절차

출력 크기와 Dynamic Resolution을 고정합니다. TSR을 유지한 채 Screen Percentage 100과 67을 비교합니다. 같은 경로의 글자·철망·반사를 녹화하고 전체 GPU 시간과 TSR 시간을 따로 기록합니다.

예상 결과와 이유

67%는 가로·세로 각각의 비율이므로 입력 픽셀 수는 약 45%가 됩니다. 화면 단위 작업은 줄 수 있지만 재구성 비용과 고정 비용 때문에 전체 시간이 같은 비율로 줄지는 않을 것입니다.

결과를 어떻게 해석할까

정지 상태의 선명도와 이동·가림 해제의 품질을 나눠 판정합니다. TSR 패스가 늘거나 유지돼도 다른 패스의 절감으로 전체가 빨라질 수 있습니다.

실제 제작에서의 선택

목표 기기에서 허용할 이동 화질과 총 GPU 시간을 기준으로 내부 해상도를 정합니다.

이해한 뒤 확인할 조건

외부 비교의 장비·장면 성능을 자신의 예상 이득으로 사용하지 않습니다. TAAU와 다른 업스케일러는 별도 비교 대상입니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[26] Anti-Aliasing and Upscaling

[27] Temporal Super Resolution

25 CPU와 GPU 병목

중심 질문

폴리곤과 해상도를 줄였는데 왜 FPS가 그대로일까?

먼저 알아둘 답

프레임 진행을 제한하는 다른 작업이 남아 있으면 일부 GPU 일을 줄여도 최종 속도는 오르지 않을 수 있습니다.

왜 알아야 할까

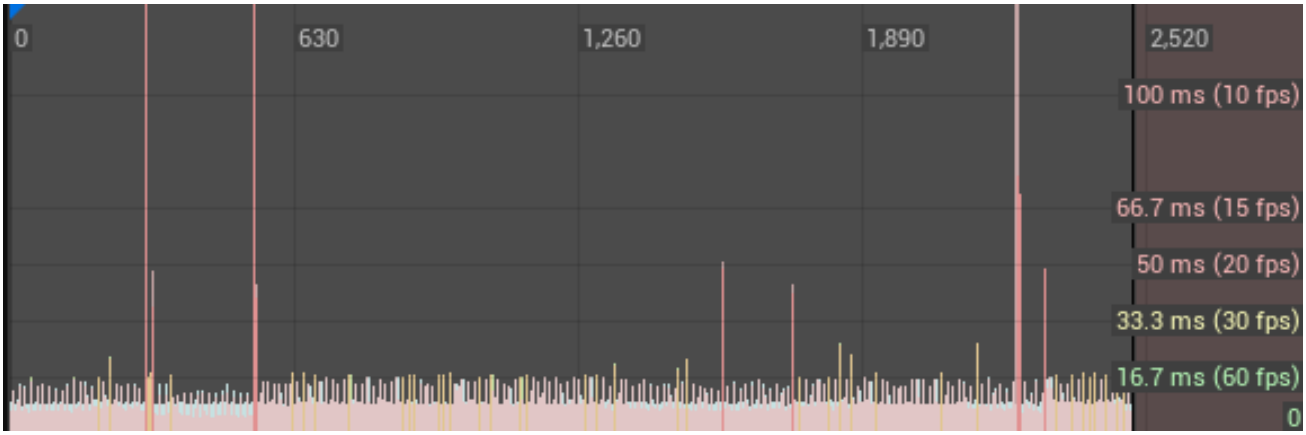
최적화 대상을 추측으로 고르지 않고 실제로 프레임을 제한하는 구간부터 수정하기 위해 배웁니다.

왜 이런 결과가 생길까

CPU는 게임 상태와 그리기 명령을 준비하고 GPU는 제출된 작업을 실행합니다. 이 작업은 여러 프레임에 걸쳐 겹칠 수 있습니다. 처리율은 느린 준비·실행 구간이나 동기화에 제한됩니다. 병목은 이 제한 원인입니다. CPU가 명령을 늦게 주면 GPU가 가벼워져도 다음 프레임 준비가 빨라지지는 않습니다.

제작 중에는 이렇게 나타납니다

최적화는 눈에 복잡한 물체를 줄이는 일과 같지 않습니다. 프레임 제한이나 VSync로 기다리는 조건이라면 계산을 줄여도 표시 FPS가 유지될 수 있습니다.



외부 실측 화면 · John Farrow가 기록한 Unreal Insights의 프레임 시간 그래프입니다.

긴 막대가 나타난 구간을 고른 뒤 CPU·GPU 시간축을 열어 원인을 찾습니다. 그래프 자체로 CPU 병목이라고 판정하지 않습니다. 저자의 2024년 프로젝트 예시이며 이 저장소의 측정 결과가 아닙니다.

© John Farrow · Unrealcode.net · 출처 [59]
원본 전체 유지 · 투명 배경은 흰색으로 표시 · 원저작권과 원문 출처 표시는 유지합니다

예상한 결과를 장면에서 확인하기

이 예제의 목표

내부 해상도 변화에 Frame과 GPU 시간이 함께 반응하는지 확인합니다.

실습 절차

진단용으로 프레임 제한과 VSync 영향을 확인합니다. 같은 경로의 안정 구간에서 stat unit의 Frame·Game·Draw·GPU를 기록한 뒤 내부 해상도만 낮춰 다시 비교합니다.

예상 결과와 이유

GPU 화면 계산이 제한 원인이면 해상도 감소에 GPU와 Frame이 함께 반응할 수 있습니다. CPU 제한이면 GPU는 줄어도 Frame 변화가 작을 수 있습니다.

결과를 어떻게 해석할까

이는 원인 후보를 좁히는 실험입니다. 가장 긴 수치가 대기 시간을 포함할 수도 있으므로 Unreal Insights에서 Game·Render·RHI·GPU 시간축을 대조해 계산과 기다림을 구분합니다.

실제 제작에서의 선택

GPU가 원인이면 무거운 패스로, CPU가 원인이면 게임 로직·명령 준비·관리 작업으로 조사 방향을 정합니다.

이해한 뒤 확인할 조건

FPS 대신 ms도 기록합니다. 프레임 제한을 풀어 원인을 진단한 뒤 실제 제품 설정으로 되돌려 효과를 확인합니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

26 GPU Profiler와 개선 검증

중심 질문

무거운 GPU 패스를 찾은 다음 어떻게 수정 효과를 증명할까?

먼저 알아둘 답

같은 조건에서 한 원인만 바꾸고, 해당 패스·전체 프레임·화질이 함께 어떻게 달라졌는지 확인합니다.

왜 알아야 할까

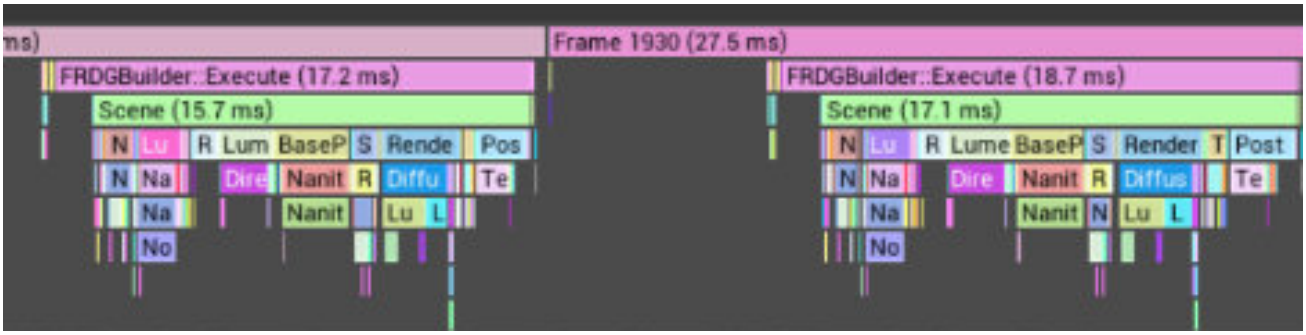
진단 도구의 숫자를 실제 제작 결정과 재현 가능한 개선 기록으로 연결하기 위해 배웁니다.

왜 이런 결과가 생길까

GPU Profiler는 작업 시간을 이벤트와 렌더 패스별로 보여 줍니다. 큰 구간 안에 세부 구간이 들어 있으므로 부모와 자식 시간을 더하면 중복할 수 있습니다. 무거운 패스를 찾은 뒤 어떤 에셋·기능이 그 일을 만드는지 가설을 세웁니다. 패스 이름만 보고 모든 그림자나 모든 반투명을 제거하는 것은 원인 검증이 아닙니다.

제작 중에는 이렇게 나타납니다

패스가 빨라져도 CPU나 다른 GPU 일이 제한하면 전체 프레임은 그대로일 수 있습니다. 반대로 시간을 줄이면서 필요한 외관을 잃었다면 목표를 만족한 개선으로 볼 수 없습니다.



외부 실제 도구 화면 · GPU Insights 시간축의 이벤트 계층을 발췌·확대했습니다.

큰 Frame 구간 안에 작은 작업 구간이 포함됩니다. 로그에서 Inclusive는 하위 작업을 포함한 시간, Exclusive는 자체 시간입니다. 부모·자식 시간을 모두 더하지 않습니다. UE 5.6 예시이므로 실제 UI와 기록 항목은 현재 빌드에서 확인합니다.

Tom Looman 문서 수록 · Unreal Engine 도구 화면 · 출처 [60]

원본 일부 발췌·확대 · 색상 유지 · 표제 추가 · 원저작권과 원문 출처 표시는 유지합니다

예상한 결과를 장면에서 확인하기

이 예제의 목표

하나의 가설을 변경·복원하며 패스 감소와 총 프레임 개선을 구분합니다.

실습 절차

CH25에서 GPU 제한을 확인한 구간을 씁니다. 예를 들어 풀의 WPO가 그림자 갱신을 늘린다는 가설을 세우고 WPO만 끕니다. 동일 경로를 워밍업 뒤 세 번씩 기록하고 원래 설정으로 돌려 재현성을 확인합니다.

예상 결과와 이유

가설이 맞으면 그림자 생성 구간이 줄고, GPU 제한이 충분하면 총 시간도 줄 것입니다. 동시에 움직이는 풀과 그림자의 외관 차이가 남을 것입니다.

결과를 어떻게 해석할까

해당 구간만 줄었으면 국소 효과를 확인한 것입니다. 전체 시간이 그대로면 다음 제한 원인을 조사합니다. 변화가 반복되지 않으면 셰이더 준비·로딩·측정 흔들림을 먼저 확인합니다.

실제 제작에서의 선택

변경 이유, 유지한 조건, 전후 시간, 외관 손실, 채택 여부를 함께 남깁니다. 실제 제품의 목표를 만족한 변경만 채택합니다.

이해한 뒤 확인할 조건

이벤트 표시와 캡처 방법은 UE 버전에 따라 다릅니다. stat gpu로 큰 범위를 보고 profilegpu 또는 Unreal Insights로 좁히며, 중첩·동시 실행 시간을 단순 합산하지 않습니다.

이어 읽을 공식 자료

구현 조건과 옵션은 아래 원문에서 확인합니다.

[03] Introduction to Performance Profiling and Configuration

[28] Stat Commands

[29] Beyond Print String · GPU Profiler

종합 실습 원인에서 제작 결정까지

질문을 좁힙니다

골목 전체를 최적화한다는 목표 대신 특정 카메라 구간이 느린 이유를 묻습니다. 이동 경로를 고정하고, CH25의 방법으로 CPU·GPU 중 제한 원인을 먼저 좁힙니다.

예상을 적습니다

GPU의 그림자 생성이 무겁고 풀 주변 페이지가 계속 갱신된다면 WPO가 원인 후보입니다. 바람만 끄면 페이지 갱신과 해당 구간 시간이 줄 것이라고 먼저 적습니다. CH22의 원리를 사용한 가설입니다.

한 번에 하나를 바꿉니다

해상도·조명·카메라는 유지하고 바람만 끕니다. 진단 시각화로 갱신 상태를 보고, 시각화를 끈 뒤 동일 경로를 반복 측정합니다. 변경을 되돌려 결과가 재현되는지도 확인합니다.

이득과 손실을 함께 읽습니다

패스 시간 감소, 총 프레임 변화, 풀과 그림자 움직임의 손실을 적습니다. 시간이 줄어도 필요한 근거리 외관을 잃었다면 그대로 채택하지 않습니다. 먼 구간의 바람만 줄이는 대안을 다시 시험합니다.

다른 장면으로 옮깁니다

배운 것은 바람을 끄라는 규칙이 아닙니다. 변하는 형상이 캐시 재사용을 줄인다는 원리를 알고, 읽히는 움직임과 갱신 비용을 거리별로 배분하는 판단입니다.

용어집

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

CPU와 GPU

CPU는 게임 상태와 작업 준비 등을 처리하는 범용 프로세서, GPU는 많은 그래픽 계산을 병렬로 처리하는 프로세서입니다. [CH01·25]

스레드

프로그램의 명령을 이어 실행하는 흐름입니다. 렌더 스레드가 CPU에서 일한다는 점과 GPU의 실행을 구분합니다. [CH01·25]

렌더 스레드

주로 CPU에서 장면의 가시성과 그릴 명령을 준비하는 실행 흐름입니다. [CH01·25]

RHI와 그래픽 API

RHI는 엔진의 그래픽 명령을 플랫폼별 그래픽 API에 연결하는 계층입니다. API는 기능을 사용하기 위해 정한 요청 방식입니다. [CH01·25]

프레임과 렌더 패스

프레임은 화면 한 장을 만드는 처리 구간이며, 패스는 그 안에서 특정 입력과 결과를 다루는 작업 단위입니다. [CH01·26]

지오메트리와 메시

지오메트리는 형태와 위치에 관한 기하 정보입니다. 이 문서의 메시는 주로 삼각형으로 구성한 3D 표면 형상입니다. [CH01·17]

정점

삼각형의 꼭짓점입니다. 위치 외에 표면 방향과 텍스처 좌표 같은 정보도 가질 수 있습니다. [CH01·10]

좌표 변환과 투영

모델 내부의 위치를 장면·카메라 기준으로 바꾸고 화면 위치에 대응시키는 과정입니다. [CH01]

셰이더

GPU에서 실행되는 계산 프로그램입니다. 정점과 화면의 표면 값 등을 처리하는 여러 종류가 있습니다. [CH01·10]

래스터화

화면에 투영한 삼각형이 어느 화면 위치를 덮는지 판정하는 과정입니다. [CH01]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

샘플과 프래그먼트

래스터 문맥의 샘플은 커버리지를 검사하는 화면 위치입니다. 프래그먼트는 삼각형에서 생긴 화면 위치별 처리 후보입니다. [CH01·05]

보간

알고 있는 값 사이에서 중간 위치의 값을 계산하는 것입니다. 삼각형 내부의 UV 등을 구하는 데 쓰입니다. [CH01]

UV와 텍스처 샘플

UV는 표면을 이미지의 위치에 대응시키는 2D 좌표입니다. 텍스처 샘플은 그 좌표에서 이미지 값을 읽는 작업입니다. [CH01·19]

법선과 노멀 맵

법선은 표면 방향입니다. 노멀 맵은 작은 표면 방향 변화를 이미지로 표현해 빛 반응에 사용합니다. [CH03·07]

탄젠트

표면을 따라 놓인 방향 기준입니다. 탄젠트 공간 노멀 맵은 이 로컬 방향 기준을 이용해 표면 방향 변화를 표현합니다. [CH03]

버퍼와 렌더 타깃

버퍼는 계산 데이터를 보관하는 메모리 공간입니다. 렌더 타깃은 그리기 결과를 기록할 이미지 자원입니다. [CH01·03]

GBuffer와 Base Pass

GBuffer는 조명에 필요한 표면 정보 묶음입니다. Base Pass는 주로 불투명 표면 정보를 준비하는 작업입니다. [CH03]

DBuffer와 데칼

데칼은 다른 표면에 투영하는 얼룩·표식 등의 표현입니다. DBuffer는 Base Pass 전에 그 표면 수정 정보를 준비합니다. [CH04]

깊이 버퍼와 Early Z

깊이 버퍼는 앞쪽 표면 깊이를 저장합니다. Early Z는 가능한 경우 이 정보로 가려진 후보를 일찍 제외합니다. [CH05]

Opaque와 Masked와 Translucent

각각 불투명, 위치별로 남길지 버리는 구멍 마스크, 뒤의 색과 합성하는 반투명 표면을 가리킵니다. [CH06·12]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

Overdraw와 커버리지

Overdraw는 같은 화면 위치의 반복 처리입니다. 커버리지는 도형이 화면의 어느 범위를 덮는지를 뜻합니다. [CH06]

PBR

재료와 빛의 상호작용을 물리적으로 그럴듯하게 근사하는 제작 방식입니다. [CH07]

Base Color와 Metallic과 Roughness

재질의 기본 색, 금속성, 반사가 퍼지는 정도를 제어하는 입력입니다. 최종 화면 색과 역할이 다릅니다. [CH07]

Material Instance와 파라미터

인스턴스는 원본 재질 구조를 공유하는 설정값 변형입니다. 파라미터는 바꿀 수 있도록 노출한 입력값입니다. [CH07·10]

BRDF와 BSDF

BRDF는 방향에 따른 반사 반응을 기술합니다. BSDF는 표면을 통한 투과까지 포괄하는 더 넓은 개념입니다. [CH08]

Fresnel과 F0

Fresnel은 각도에 따른 계면 반사율의 변화입니다. F0는 법선 방향 입사의 반사율입니다. [CH09]

컴파일과 셰이더 순열

컴파일은 실행할 프로그램을 준비하는 과정입니다. 순열은 기능·정적 설정 등의 조합에 따른 실행 버전을 뜻합니다. [CH10]

Static Switch와 WPO

Static Switch는 컴파일할 때 경로를 고릅니다. WPO는 정점의 월드 위치에 변화를 더하는 재질 입력입니다. [CH10]

캐시와 대역폭

캐시는 데이터를 다시 읽기 쉽게 두는 임시 저장 영역입니다. 대역폭은 일정 시간에 옮길 수 있는 데이터 양입니다. [CH03·10]

Slab과 Closure

Slab은 Substrate의 재료 정의 단위입니다. Closure는 실행 시 조명에 대해 평가할 재료 반응의 단위를 가리킵니다. [CH11]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

파라미터 블렌딩

복수 재료의 설정값을 하나의 재료 반응으로 합쳐 근사하는 방식입니다. [CH11]

알파 합성과 굴절

알파 합성은 앞뒤 색을 비중에 따라 섞는 계산입니다. 굴절은 물질 경계에서 빛 방향이 바뀌는 현상입니다. [CH12]

SSS와 MFP

SSS는 표면 아래 빛의 산란입니다. MFP는 내부 상호작용 사이의 평균 거리 스케일을 가리킵니다. [CH13]

Actor와 컴포넌트

Actor는 레벨에 배치하는 오브젝트 단위입니다. 컴포넌트는 그 안의 메시·조명 등 기능 요소입니다. [CH14·15]

Draw Call과 메시 섹션

Draw Call은 GPU에 그리기를 제출하는 명령입니다. 섹션은 보통 재질을 따로 적용할 수 있는 메시의 구역입니다. [CH14]

배칭과 인스턴싱

배칭은 호환되는 작업을 묶는 접근입니다. 인스턴싱은 메시 등의 데이터를 공유해 반복 배치를 처리하는 방식입니다. [CH14·15]

ISM과 HISM

반복 Static Mesh를 컴포넌트 안에서 관리하는 기능입니다. HISM은 공간적으로 묶은 계층을 활용합니다. [CH15]

LOD와 FOV

LOD는 필요한 화면 디테일에 맞춘 대체 표현입니다. FOV는 카메라가 담는 시야각입니다. [CH16]

클러스터와 폴백 메시

Nanite 클러스터는 작은 삼각형 묶음입니다. 폴백 메시는 다른 기능이나 조건에서 사용하는 대체 표현입니다. [CH17]

컬링과 bounds

컬링은 불필요한 가시 후보를 제외하는 과정입니다. bounds는 이런 판정 등에 쓰는 물체를 둘러싼 범위입니다. [CH18]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

밉맵과 필터링

밉맵은 축소 이미지의 계층입니다. 필터링은 필요한 범위의 값을 읽고 섞어 화면에 대응시키는 처리입니다. [CH19]

스트리밍 풀과 I/O

풀은 관리할 메모리 예산의 한 묶음입니다. I/O는 디스크 등에서 데이터를 읽고 옮기는 입출력 작업입니다. [CH20]

Screen Trace와 거리장

Screen Trace는 화면 정보로 경로를 조사합니다. 거리장은 표면까지의 거리를 저장한 장면 표현입니다. [CH21]

Surface Cache와 Cards

Surface Cache는 표면 정보를 빠르게 조회하는 체계입니다. Cards는 표면을 여러 방향에서 담는 표현에 사용됩니다. [CH21]

레이 트레이싱과 가속 구조

광선과 물체의 교차를 추적하는 계산과, 교차 후보를 빨리 찾도록 준비한 자료 구조입니다. [CH21]

VSM 페이지와 캐스터

페이지는 가상 그림자 맵의 작은 영역입니다. 캐스터는 그림자를 만드는 물체입니다. [CH22]

캐시 무효화

예전에 계산해 둔 결과가 현재 장면에 맞지 않아 다시 계산해야 하는 상태입니다. [CH22]

히스토리와 지터

히스토리는 이전 프레임의 저장 정보입니다. 지터는 프레임마다 샘플 위치를 조금씩 바꾸는 동작입니다. [CH23]

모션 벡터와 재투영

모션 벡터는 화면상 이동 정보입니다. 재투영은 과거 정보를 현재 화면 위치에 대응시키는 과정입니다. [CH23]

고스팅

과거 정보가 잘못 남아 움직이는 표면 뒤로 끌리거나 번져 보이는 시간적 아티팩트입니다. [CH23]

용어집 계속

원리를 다시 확인하려면 표시한 챕터로 돌아갑니다.

내부 해상도와 업스케일러

내부 해상도는 주요 장면 계산의 크기입니다. 업스케일러는 그 결과 등을 이용해 더 큰 출력 화면을 만드는 기능입니다. [CH24]

병목과 VSync

병목은 처리율을 제한하는 원인입니다. VSync는 디스플레이 갱신에 출력 시점을 맞추는 기능입니다. [CH25]

중앙값과 P95

중앙값은 크기순 가운데 값입니다. P95는 측정값의 95%가 그 이하에 놓이는 경계로, 느린 프레임을 살필 때 사용합니다. [CH25·26]

프로파일링과 워밍업

프로파일링은 실행 중의 비용 측정입니다. 워밍업은 셰이더 준비·로딩 등을 끝내 안정된 비교 조건을 만드는 과정입니다. [CH26]

톤매핑과 Bloom

톤매핑은 장면 밝기를 표시할 범위로 옮기는 처리입니다. Bloom은 밝은 부분이 주변으로 번지는 화면 효과입니다. [CH01]

Scalability와 쿠킹

Scalability는 엔진의 품질 설정 묶음입니다. 쿠킹은 대상 플랫폼에서 실행할 배포용 데이터를 준비하는 과정입니다. [CH26]

심화와 적용 조건

1 스레드와 파이프라인 도식의 범위

RHI는 인터페이스 계층의 이름입니다. RHI 작업이 별도 스레드에서 처리되는 방식과 병렬화 구성은 플랫폼·설정에 따라 달라집니다. 본문의 단계 그림은 작업 의존 관계를 이해하기 위한 것입니다. 실제 GPU 프레임은 그림자·추적·후처리 등의 분기와 동시 실행을 포함하므로 GPU 캡처에서 확인합니다.

2 BRDF 수식을 더 읽을 때

대표적인 미세면 스펙큘러 항은 D×F×G를 4(N·L)(N·V)로 나눈 형태로 설명할 수 있습니다. D는 미세면 방향 분포, F는 Fresnel 반사율, G는 미세면 사이의 가림입니다. N은 표면 법선, L은 빛 방향, V는 시선 방향이며 점 기호는 두 방향의 관계를 계산하는 내적입니다. 실제 UE 재질의 모든 경로가 이 한 식과 같은 구현이라는 뜻은 아닙니다.

3 Substrate 저장 형식과 버전

이 문서는 UE 5.8 문서와 실습 프로젝트의 5.8.4를 기준으로 합니다. Adaptive GBuffer의 지원 플랫폼과 단순화 조건, DBuffer 요구 사항은 해당 버전의 공식 문서에서 확인합니다. 60Hz 지향은 설계 목표이며 실제 게임의 60 FPS 달성을 보장하는 수치가 아닙니다. 프로젝트 범위 변경은 재시작이나 셰이더 재컴파일을 요구할 수 있습니다.

4 GPU 프로파일러의 표시와 시간 합계

UE 5.6 이후 구성에서는 구형 GPU Visualizer 팝업과 같은 표시를 전제로 하지 않고 로그와 Unreal Insights의 GPU 트랙을 확인합니다. 중첩 이벤트, 동시 실행과 CPU·GPU의 대기 때문에 항목의 단순 합이 총 프레임과 다를 수 있습니다. 공식 도구 설명과 대상 빌드의 실제 동작을 함께 확인합니다.

5 셰이더 실행과 프로그램 준비

머티리얼 그래프는 컴파일을 거쳐 실행 프로그램이 됩니다. 정적 스위치 등은 서로 다른 프로그램 조합인 셰이더 순열을 만들 수 있습니다. 실행 시간, 컴파일 대기, 쿠킹·배포 준비는 다른 지표입니다. CH10의 WPO 실험으로 세 지표를 한 번에 검증하지 않습니다.

출처 목록

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

01 Rendering Settings

02 Render Dependency Graph

03 Introduction to Performance Profiling and Configuration

04 Forward Shading Renderer

05 Programming with Substrate GBuffer Formats

06 Decal Materials

07 Viewport Modes

08 Physically Based Materials

09 Using Fresnel

10 Overview of Substrate Materials

11 Material Editor UI

12 Material Inputs

13 Lit Translucency

14 Using Refraction

15 Subsurface Profile Shading Model

16 Mesh Drawing Pipeline

17 Instanced Static Mesh Component

18 Optimizing LOD Screen Size Per Platform

출처 목록 계속

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

19 Nanite Virtualized Geometry

20 Texture Asset Editor

21 Texture Streaming Overview

22 Texture Streaming Metrics

23 Lumen Technical Details

24 Lumen Performance Guide

25 Virtual Shadow Maps

26 Anti-Aliasing and Upscaling

27 Temporal Super Resolution

28 Stat Commands

29 Beyond Print String · GPU Profiler

30 Visibility and Occlusion Culling

31 Real Shading in Unreal Engine 4 · Brian Karis

32 Scripting the Editor Using Python

33 Parallel Rendering Overview

34 Direct3D Rasterizer Stage

35 Material Instances

36 Cornellbox pathtracing · Trevorgoodchild · Public domain

출처 목록 계속

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

37 LearnOpenGL Coordinate Systems · 이미지 출처

38 LearnOpenGL Anti Aliasing · 이미지 출처

39 LearnOpenGL Deferred Shading · 이미지 출처

40 LearnOpenGL PBR Lighting · 이미지 출처

41 LearnOpenGL PBR Theory · 이미지 출처

42 LOD Example · Sirotk · CC BY-SA 3.0

43 Mipmap Aliasing Comparison · BillyBob CornCob · CC0

44 LearnOpenGL 이미지 이용 조건 · CC BY 4.0

45 Epic Nanite Preserve Area 비교 이미지 · UE 5.5

46 Decal Materials 비교 이미지 · UE 5.5

47 LearnOpenGL Depth Testing 비교 이미지

48 Android Developers Materials and Shaders

49 Substrate 코팅 비교 이미지 · UE 5.4

50 Subsurface 비교 이미지 · UE 5.5

51 Mesh Drawing Pipeline 도식 · UE 5.5

52 LearnOpenGL Instancing 렌더 예시

53 Visibility and Occlusion Culling 단계별 그림

54 Texture Streaming Metrics 통계 화면 · UE 5.5

출처 목록 계속

본문의 공식 링크와 외부 이미지 원문을 함께 보관합니다. 모든 자료가 본문의 측정 증거라는 뜻은 아닙니다.

55 Lumen 반사 조명 비교 이미지 · UE 5.5

56 VSM 캐시와 무효화 범위 · UE 5.4

57 Anti Aliasing 전후 이미지 · UE 5.5

58 TSR 동일 출력 비교 · UE 5.5

59 Unreal Insights 실측 화면 · John Farrow

60 GPU Insights와 ProfileGPU 화면 · Tom Looman

61 New GPU Profiler and RHI Submission Pipeline · Epic